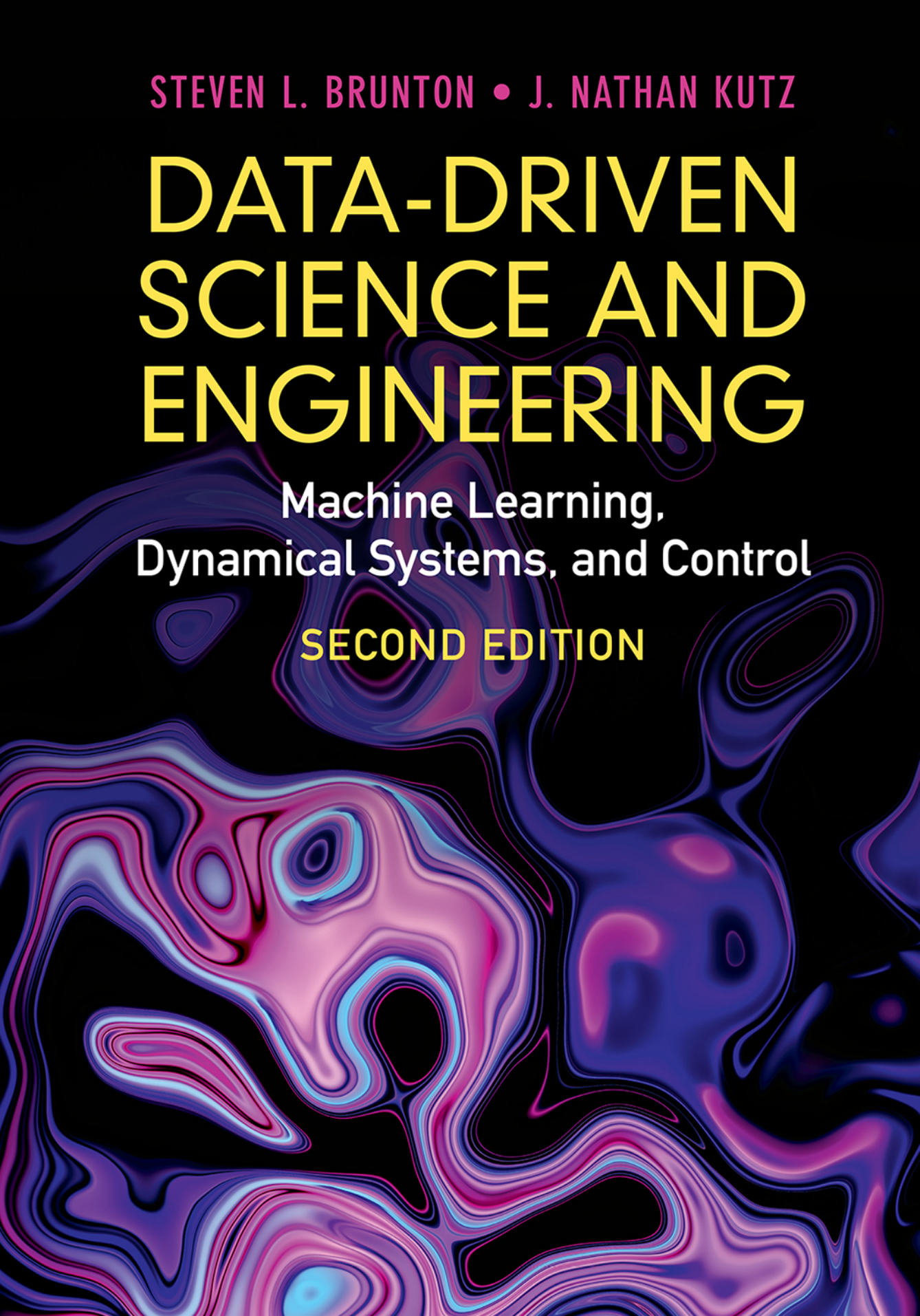


STEVEN L. BRUNTON • J. NATHAN KUTZ

DATA-DRIVEN SCIENCE AND ENGINEERING

Machine Learning,
Dynamical Systems, and Control

SECOND EDITION



Data-Driven Science and Engineering

Data-driven discovery is revolutionizing how we model, predict, and control complex systems. Now with Python and MATLAB®, this textbook trains mathematical scientists and engineers for the next generation of scientific discovery by offering a broad overview of the growing intersection of data-driven methods, machine learning, applied optimization, and the classic fields of engineering mathematics and mathematical physics. With a focus on integrating dynamical systems modeling and control with modern methods in applied machine learning, this text includes methods that were chosen for their relevance, simplicity, and generality. Topics range from introductory to research-level material, making it accessible to advanced undergraduate and beginning graduate students from the engineering and physical sciences. This second edition features new chapters on reinforcement learning and physics-informed machine learning, significant new sections throughout, and chapter exercises. Online supplementary material – including lecture videos per section, homework, data, and codes in MATLAB®, Python, and R – is available on <http://databookuw.com>.

Steven L. Brunton is the James B. Morrison Professor of Mechanical Engineering at the University of Washington (UW) and Associate Director of the NSF AI Institute in Dynamic Systems. He is also Adjunct Professor of Applied Mathematics and Computer Science, and a Data-Science Fellow at the eScience Institute. His research merges machine learning with dynamical systems and control, with applications in fluid dynamics, bio-locomotion, optics, energy systems, and manufacturing. He is an author of three textbooks, and received the UW College of Engineering Teaching Award, the Army and Air Force Young Investigator Program (YIP) Awards, and the Presidential Early Career Award for Scientists and Engineers (PECASE).

J. Nathan Kutz is the Robert Bolles and Yasuko Endo Professor of Applied Mathematics at the University of Washington (UW) and Director of the NSF AI Institute in Dynamic Systems. He is also Adjunct Professor of Electrical and Computer Engineering, Mechanical Engineering, and Physics, and Senior Data-Science Fellow at the eScience Institute. His research interests lie at the intersection of dynamical systems and machine learning. He is an author of three textbooks and has received the Applied Mathematics Boeing Award of Excellence in Teaching and an NSF CAREER Award.

“Finally, a book that introduces data science in a context that will make any mechanical engineer feel comfortable. Data science is the new calculus, and no engineer should graduate without a thorough understanding of the topic”

Hod Lipson, Columbia University

“This book is a must-have for anyone interested in data-driven modeling and simulations. Readers as diverse as undergraduate STEM students and seasoned researchers would find it useful as a guide to this rapidly evolving field. Topics covered by the monograph include dimension reduction, machine learning, and robust control of dynamical systems with uncertain/random inputs. Every chapter contains codes and homework problems, which make this treatise ideal for the classroom setting. The book is supplemented with online lectures, which are not only educational but also entertaining to watch.”

Daniel M. Tartakovsky, Stanford University

“Engineering principles will always be based on physics, and the models that underpin engineering will be derived from these physical laws. But, in the future, models based on relationships in large datasets will be as important and, when used alongside physics-based models, will lead to new insights and designs. Brunton and Kutz will equip students and practitioners with the tools they will need for this exciting future.”

Greg Hyslop, Boeing

“Brunton and Kutz’s book is fast becoming an indispensable resource for machine learning and data-driven learning in science and engineering. The second edition adds several timely topics in this lively field, including reinforcement learning and physics-informed machine learning. The text balances theoretical foundations and concrete examples with code, making it accessible and practical for students and practitioners alike.”

Tim Colonius, California Institute of Technology

“This is a must-read for those who are interested in understanding what machine learning can do for dynamical systems! Steve and Nathan have done an excellent job in bringing everyone up to speed with the modern application of machine learning to these complex dynamical systems.”

Shirley Ho, Flatiron Institute and New York University

Data-Driven Science and Engineering

**Machine Learning, Dynamical Systems,
and Control**

Second Edition

STEVEN L. BRUNTON

University of Washington

J. NATHAN KUTZ

University of Washington



CAMBRIDGE
UNIVERSITY PRESS

CAMBRIDGE
UNIVERSITY PRESS

University Printing House, Cambridge CB2 8BS, United Kingdom

One Liberty Plaza, 20th Floor, New York, NY 10006, USA

477 Williamstown Road, Port Melbourne, VIC 3207, Australia

314–321, 3rd Floor, Plot 3, Splendor Forum, Jasola District Centre,
New Delhi – 110025, India

103 Penang Road, #05–06/07, Visioncrest Commercial, Singapore 238467

Cambridge University Press is part of the University of Cambridge.

It furthers the University's mission by disseminating knowledge in the pursuit of
education, learning, and research at the highest international levels of excellence.

www.cambridge.org

Information on this title: www.cambridge.org/highereducation/isbn/9781009098489

DOI: [10.1017/9781009098489](https://doi.org/10.1017/9781009098489)

© Steven L. Brunton and J. Nathan Kutz 2022

This publication is in copyright. Subject to statutory exception
and to the provisions of relevant collective licensing agreements,
no reproduction of any part may take place without the written
permission of Cambridge University Press.

First published 2019

Second edition 2022

Printed in the United Kingdom by TJ Books Limited, Padstow, Cornwall

A catalogue record for this publication is available from the British Library.

ISBN 978-1-009-09848-9 Hardback

Cambridge University Press has no responsibility for the persistence or accuracy
of URLs for external or third-party internet websites referred to in this publication
and does not guarantee that any content on such websites is, or will remain,
accurate or appropriate.

Contents

<i>Preface</i>	<i>page ix</i>
<i>Acknowledgments</i>	<i>xiv</i>
<i>Common Optimization Techniques, Equations, Symbols, and Acronyms</i>	<i>xv</i>

Part I Dimensionality Reduction and Transforms	1
1 Singular Value Decomposition (SVD)	3
1.1 Overview	3
1.2 Matrix Approximation	7
1.3 Mathematical Properties and Manipulations	12
1.4 Pseudo-Inverse, Least-Squares, and Regression	16
1.5 Principal Component Analysis (PCA)	23
1.6 Eigenfaces Example	28
1.7 Truncation and Alignment	35
1.8 Randomized Singular Value Decomposition	40
1.9 Tensor Decompositions and N -Way Data Arrays	46
2 Fourier and Wavelet Transforms	53
2.1 Fourier Series and Fourier Transforms	53
2.2 Discrete Fourier Transform (DFT) and Fast Fourier Transform (FFT)	63
2.3 Transforming Partial Differential Equations	70
2.4 Gabor Transform and the Spectrogram	76
2.5 Laplace Transform	81
2.6 Wavelets and Multi-Resolution Analysis	85
2.7 Two-Dimensional Transforms and Image Processing	87
3 Sparsity and Compressed Sensing	97
3.1 Sparsity and Compression	97
3.2 Compressed Sensing	101
3.3 Compressed Sensing Examples	105
3.4 The Geometry of Compression	109
3.5 Sparse Regression	113
3.6 Sparse Representation	117
3.7 Robust Principal Component Analysis (RPCA)	120
3.8 Sparse Sensor Placement	123

Part II Machine Learning and Data Analysis	131
4 Regression and Model Selection	133
4.1 Classic Curve Fitting	134
4.2 Nonlinear Regression and Gradient Descent	140
4.3 Regression and $Ax = b$: Over- and Under-Determined Systems	145
4.4 Optimization as the Cornerstone of Regression	151
4.5 The Pareto Front and <i>Lex Parsimoniae</i>	155
4.6 Model Selection: Cross-Validation	158
4.7 Model Selection: Information Criteria	162
5 Clustering and Classification	168
5.1 Feature Selection and Data Mining	169
5.2 Supervised versus Unsupervised Learning	174
5.3 Unsupervised Learning: k -Means Clustering	178
5.4 Unsupervised Hierarchical Clustering: Dendrogram	182
5.5 Mixture Models and the Expectation-Maximization Algorithm	186
5.6 Supervised Learning and Linear Discriminants	189
5.7 Support Vector Machines (SVM)	193
5.8 Classification Trees and Random Forest	198
5.9 Top 10 Algorithms of Data Mining circa 2008 (Before the Deep Learning Revolution)	203
6 Neural Networks and Deep Learning	208
6.1 Neural Networks: Single-Layer Networks	209
6.2 Multi-Layer Networks and Activation Functions	214
6.3 The Backpropagation Algorithm	219
6.4 The Stochastic Gradient Descent Algorithm	222
6.5 Deep Convolutional Neural Networks	224
6.6 Neural Networks for Dynamical Systems	228
6.7 Recurrent Neural Networks	233
6.8 Autoencoders	236
6.9 Generative Adversarial Networks (GANs)	240
6.10 The Diversity of Neural Networks	242
Part III Dynamics and Control	251
7 Data-Driven Dynamical Systems	253
7.1 Overview, Motivations, and Challenges	254
7.2 Dynamic Mode Decomposition (DMD)	260
7.3 Sparse Identification of Nonlinear Dynamics (SINDy)	275
7.4 Koopman Operator Theory	286
7.5 Data-Driven Koopman Analysis	296

8	Linear Control Theory	311
	8.1 Closed-Loop Feedback Control	312
	8.2 Linear Time-Invariant Systems	317
	8.3 Controllability and Observability	322
	8.4 Optimal Full-State Control: Linear–Quadratic Regulator (LQR)	328
	8.5 Optimal Full-State Estimation: the Kalman Filter	332
	8.6 Optimal Sensor-Based Control: Linear–Quadratic Gaussian (LQG)	335
	8.7 Case Study: Inverted Pendulum on a Cart	336
	8.8 Robust Control and Frequency-Domain Techniques	346
9	Balanced Models for Control	360
	9.1 Model Reduction and System Identification	360
	9.2 Balanced Model Reduction	361
	9.3 System Identification	375
	Part IV Advanced Data-Driven Modeling and Control	387
10	Data-Driven Control	389
	10.1 Model Predictive Control (MPC)	390
	10.2 Nonlinear System Identification for Control	392
	10.3 Machine Learning Control	398
	10.4 Adaptive Extremum-Seeking Control	408
11	Reinforcement Learning	419
	11.1 Overview and Mathematical Formulation	419
	11.2 Model-Based Optimization and Control	426
	11.3 Model-Free Reinforcement Learning and Q -Learning	429
	11.4 Deep Reinforcement Learning	436
	11.5 Applications and Environments	440
	11.6 Optimal Nonlinear Control	444
12	Reduced-Order Models (ROMs)	449
	12.1 Proper Orthogonal Decomposition (POD) for Partial Differential Equations	449
	12.2 Optimal Basis Elements: the POD Expansion	455
	12.3 POD and Soliton Dynamics	461
	12.4 Continuous Formulation of POD	465
	12.5 POD with Symmetries: Rotations and Translations	470
	12.6 Neural Networks for Time-Stepping with POD	475
	12.7 Leveraging DMD and SINDy for Galerkin–POD	479
13	Interpolation for Parametric Reduced-Order Models	485
	13.1 Gappy POD	485
	13.2 Error and Convergence of Gappy POD	490
	13.3 Gappy Measurements: Minimize Condition Number	493
	13.4 Gappy Measurements: Maximal Variance	497

13.5	POD and the Discrete Empirical Interpolation Method (DEIM)	500
13.6	DEIM Algorithm Implementation	504
13.7	Decoder Networks for Interpolation	508
13.8	Randomization and Compression for ROMs	512
13.9	Machine Learning ROMs	513
14	Physics-Informed Machine Learning	520
14.1	Mathematical Foundations	520
14.2	SINDy Autoencoder: Coordinates and Dynamics	523
14.3	Koopman Forecasting	526
14.4	Learning Nonlinear Operators	529
14.5	Physics-Informed Neural Networks (PINNs)	533
14.6	Learning Coarse-Graining for PDEs	535
14.7	Deep Learning and Boundary Value Problems	539
	<i>Glossary</i>	542
	<i>References</i>	552
	<i>Index</i>	588

Preface

This book is about the growing intersection of data-driven methods, machine learning, applied optimization, and the classical fields of engineering mathematics and mathematical physics. We developed this material over a number of years, primarily to educate our advanced undergraduate and beginning graduate students from engineering and physical science departments. Typically, such students have backgrounds in linear algebra, differential equations, and scientific computing, with engineers often having some exposure to control theory and/or partial differential equations. However, most undergraduate curricula in engineering and science fields have little or no exposure to data methods and/or optimization. Likewise, computer scientists and statisticians have little exposure to dynamical systems and control. Our goal is to provide a broad entry point to applied machine learning for both of these groups of students. We have chosen the methods discussed in this book for their (1) relevance, (2) simplicity, and (3) generality, and we have attempted to present a range of topics, from basic introductory material up to research-level techniques.

Data-driven discovery is currently revolutionizing how we model, predict, and control complex systems. The most pressing scientific and engineering problems of the modern era are not amenable to empirical models or derivations based on first principles. Increasingly, researchers are turning to data-driven approaches for a diverse range of complex systems, such as turbulence, the brain, climate, epidemiology, finance, robotics, and autonomy. These systems are typically nonlinear, dynamic, multi-scale in space and time, and high-dimensional, with dominant underlying patterns that should be characterized and modeled for the eventual goal of sensing, prediction, estimation, and control. With modern mathematical methods, enabled by the unprecedented availability of data and computational resources, we are now able to tackle previously unattainable problems. A small handful of these new techniques include robust image reconstruction from sparse and noisy random pixel measurements, turbulence control with machine learning, optimal sensor and actuator placement, discovering interpretable nonlinear dynamical systems purely from data, and reduced-order models to accelerate the optimization and control of systems with complex multi-scale physics.

Driving modern data science is the availability of vast and increasing quantities of data, enabled by remarkable innovations in low-cost sensors, orders-of-magnitude increases in computational power, and virtually unlimited data storage and transfer capabilities. Such vast quantities of data are affording engineers and scientists across all disciplines new opportunities for data-driven discovery, which has been referred to as the fourth paradigm of scientific discovery [325]. This fourth paradigm is the natural culmination of the

first three paradigms: empirical experimentation, analytical derivation, and computational investigation. The integration of these techniques provides a transformative framework for data-driven discovery efforts. This process of scientific discovery is not new, and indeed mimics the efforts of leading figures of the scientific revolution: Johannes Kepler (1571–1630) and Sir Isaac Newton (1642–1727). Each played a critical role in developing the theoretical underpinnings of celestial mechanics, based on a combination of empirical data-driven and analytical approaches. Data science is not replacing mathematical physics and engineering, but is instead augmenting it for the twenty-first century, resulting in more of a renaissance than a revolution.

Data science itself is not new, having been proposed more than 50 years ago by John Tukey, who envisioned the existence of a scientific effort focused on learning from data, or *data analysis* [204]. Since that time, data science has been largely dominated by two distinct cultural outlooks on data [109]. The *machine learning* community, which predominantly comprises computer scientists, is typically centered on prediction quality and scalable, fast algorithms. Although not necessarily in contrast, the *statistical learning* community, often centered in statistics departments, focuses on the inference of interpretable models. Both methodologies have achieved significant success and have provided the mathematical and computational foundations for data science methods. For engineers and scientists, the goal is to leverage these broad techniques to infer and compute models (typically nonlinear) from observations that correctly identify the underlying dynamics *and* generalize qualitatively and quantitatively to unmeasured parts of phase, parameter, or application space. Our goal in this book is to leverage the power of both statistical and machine learning to solve engineering problems.

Themes of This Book

There are a number of key themes that have emerged throughout this book. First, many complex systems exhibit *dominant low-dimensional patterns* in the data, despite the rapidly increasing resolution of measurements and computations. This underlying structure enables efficient sensing, and compact representations for modeling and control. Pattern extraction is related to the second theme of finding *coordinate transforms* that simplify the system. Indeed, the rich history of mathematical physics is centered around coordinate transformations (e.g., spectral decompositions, the Fourier transform, generalized functions, etc.), although these techniques have largely been limited to simple idealized geometries and linear dynamics. The ability to derive *data-driven* transformations opens up opportunities to generalize these techniques to new research problems with more complex geometries and boundary conditions. We also take the perspective of *dynamical systems and control* throughout the book, applying data-driven techniques to model and control systems that evolve in time. Perhaps the most pervasive theme is that of *data-driven applied optimization*, as nearly every topic discussed is related to optimization (e.g., finding *optimal* low-dimensional patterns, *optimal* sensor placement, machine learning *optimization*, *optimal* control, etc.). Even more fundamentally, most data is organized into arrays for analysis, where the extensive development of numerical linear algebra tools from the early 1960s onward provides many of the foundational mathematical underpinnings for matrix decompositions and solution strategies used throughout this text.

Overview of Second Edition

The integration of machine learning methods in science and engineering has advanced significantly in the two years since publication of the first edition. The field is fast-moving, with innovations coming in a diversity of application areas that use creative mathematical architectures for advancing the state of the art in data-driven modeling and control. This second edition is aimed at capturing some of the more salient and successful advancements in the field. It helps bring the reader to a modern understanding of what is possible using machine learning in science and engineering. As with the first edition, extensive online supplementary material can be found at the book's website:

<http://databookuw.com>

Major changes in the second edition include the following.

- **Homework:** Extensive homework has been added to every chapter, with additional homework and projects on the book's website. Homework ranges in difficulty from introductory demonstrations and concept-building to advanced problems that reproduce modern research papers and may be the basis of course projects.
- **Code:** Python code has been added throughout, in parallel to existing MATLAB code, and both sets of codes have been streamlined considerably. All extended codes are available in MATLAB and Python on the book's website and GitHub pages.

- Python Code:
https://github.com/dynamicslab/databook_python
- MATLAB Code:
https://github.com/dynamicslab/databook_matlab

Wherever possible, a minimal representation of code has been presented in the text to improve readability. These code blocks are equivalently expressed in MATLAB and Python. In more advanced examples, it is often advantageous to use either MATLAB or Python, but not both. In such cases, this has been indicated and only a single code block is demonstrated. The full code is available at the above GitHub sites as well as on the book's website. In addition, extensive codes are available in R online. We encourage the reader to read the book and follow along with code to help improve the learning process and experience.

- **New chapters:** Two new chapters have been added on “Reinforcement Learning” and “Physics-Informed Machine Learning,” which are two of the most exciting and rapidly growing fields of research in machine learning, modeling, and control.
 - Reinforcement Learning: Reinforcement learning is a third major branch of machine learning that is concerned with how to learn control laws and policies to interact with a complex environment. This is a critical area of research, situated at the growing intersection of control theory and machine learning.
 - Physics-Informed Machine Learning: The integration of physics concepts, constraints, and symmetries is providing exceptional opportunities for training machine learning algorithms that are encoded with knowledge of physics. This chapter features a number of recent innovations aimed at understanding how this can be done in principle and in practice.

- **New sections:** We have added and improved material throughout, including the following.
 - [Chapter 1](#): new sections discussing condition number, connections to the eigendecomposition, and error bounds for SVD (singular value decomposition) based approximations.
 - [Chapter 2](#): new section on the Laplace transform.
 - [Chapter 6](#): new sections devoted to autoencoders, recurrent neural networks, and generative adversarial networks.
 - [Chapter 7](#): addition of recent innovations to DMD (dynamic mode decomposition), Koopman theory, and SINDy (sparse identification of nonlinear dynamics).
 - [Chapter 10](#): new section on model predictive control.
 - [Chapter 12](#) (previously [Chapter 11](#)): new sections on using neural networks for time-stepping in reduced-order models, as well as non-intrusive methods such as DMD.
 - [Chapter 13](#) (previously [Chapter 12](#)): new sections on decoder networks for interpolation in model reduction as well as randomized linear algebra methods for scalable reduced-order models.
- **Videos:** An extensive collection of video lectures are available on YouTube, covering nearly every topic from each section of the book. Videos may be found on our YouTube channels.
 - www.youtube.com/c/eigensteve
 - www.youtube.com/c/NathanKutzAMATH
 - www.youtube.com/c/PhysicsInformedMachineLearning
- **Typos:** We have corrected typos and mistakes throughout the second edition.

Online Material

We have designed this book to make extensive use of online supplementary material, including codes, data, videos, homework, and suggested course syllabi. All of this material can be found at the book's website: <http://databookuw.com>.

In addition to course resources, all of the code and data used in the book are available on the book's GitHub: <https://github.com/dynamicslab/>. The codes online are more extensive than those presented in the book, including code used to generate publication-quality figures. In addition to the Python and MATLAB used throughout the text, online code is also available in R. Data visualization was ranked as the top-used data science method in the Kaggle 2017 *The State of Data Science and Machine Learning* study, and so we highly encourage readers to download the online codes and make full use of these plotting commands.

We have also recorded and posted video lectures on YouTube for every section in this book, available at www.youtube.com/c/eigensteve and www.youtube.com/c/NathanKutzAMATH. We include supplementary videos for students to fill in gaps in their background on scientific computing and foundational applied mathematics. We have designed this text to be both a reference as well as the material for several courses at various levels of student

preparation. Most chapters are also modular, and may be converted into stand-alone *boot camps*, containing roughly 10 hours of materials each.

How to Use This Book

Our intended audience includes beginning graduate students, or advanced undergraduates, in engineering and science. As such, the machine learning methods are introduced at a beginning level, whereas we assume students know how to model physical systems with differential equations and simulate them with solvers such as **ode45**. The diversity of topics covered thus range from introductory to state-of-the-art research methods. Our aim is to provide an integrated viewpoint and mathematical toolset for solving engineering and science problems. Alternatively, the book can also be useful for computer science and statistics students, who often have limited knowledge of dynamical systems and control. Various courses can be designed from this material, and several example syllabi may be found on the book's website – this includes homework, data sets, and code.

First and foremost, we want this book to be fun, inspiring, eye-opening, and empowering for young scientists and engineers. We have attempted to make everything as simple as possible, while still providing the depth and breadth required to be useful in research. Many of the chapter topics in this text could be entire books in their own right, and many of them are. However, we also wanted to be as comprehensive as may be reasonably expected for a field that is so big and moving so fast. We hope that you enjoy this book, master these methods, and change the world with applied data science!

Acknowledgments

We are indebted to many wonderful students, collaborators, and colleagues for valuable feedback, suggestions, and support. We are especially grateful to Joshua Proctor, who was instrumental in the origination of this book and who helped guide much of the framing and organization. We have also benefited from extensive interactions and conversations with Bing Brunton, Jean-Christophe Loiseau, Bernd Noack, and Sam Taira. This work would also not have been possible without our many great colleagues and collaborators, with whom we have worked and whose research is featured throughout this book.

Throughout the writing of the first edition and teaching of related courses, we have received great feedback and comments from our excellent students and postdocs: Travis Askham, Michael Au-Yeung, Zhe Bai, Ido Bright, Kathleen Champion, Emily Clark, Charles Delahunt, Daniel Dylewski, Ben Erichson, Charlie Fiesler, Xing Fu, Chen Gong, Taren Gorman, Jacob Grosek, Seth Hirsh, Mikala Johnson, Eurika Kaiser, Mason Kamb, James Kunert, Bethany Lusch, Pedro Maia, Niall Mangan, Krithika Manohar, Ariana Mendible, Thomas Mohren, Megan Morrison, Markus Quade, Sam Rudy, Susanna Sargsyan, Isabel Scherl, Eli Shlizerman, George Stepaniants, Ben Strom, Chang Sun, Roy Taylor, Meghana Velagar, Jake Weholt, and Matt Williams. Our students are our inspiration for this book, and they make it fun and exciting to come to work every day.

For this second edition, special thanks are in order to Daniel Dylewski for his incredible efforts generating extensive Python code for the book. We are indebted to his efforts in helping generate the bulk of this code. We also thank Richard Knight for generously sharing his R code online. We have also benefited from extensive discussions with Bing Brunton. We are also grateful to Scott Dawson for incredibly helpful, detailed, and insightful comments and errata throughout the entire book. We would also like to thank the following for contributing corrections, typos, errata, and suggestions throughout, and also to all of those we have missed: Asude Aydin, Dammalapati Harshavardhan, Zdenek Hurak, Jamie Johnson, Lance Larsen, Pietro Monticone, Matt Reeves, Joel Rosenfeld, Simon Schauppenlehner, Csaba Szepesvari, Yuchan Tseng, Balint Uveges, and Ning Zheng.

We are grateful for the continued support and encouragement of our publishers Katie Leach and Lauren Cowles at Cambridge University Press. We are especially indebted to Katie for her patient handling of this second edition, and to Charles Howell and Geoff Amor for greatly improving the book through production and copyediting.

Steven L. Brunton and J. Nathan Kutz
Seattle, WA, March 2018
Second Edition, September 2021

Common Optimization Techniques, Equations, Symbols, and Acronyms

Most Common Optimization Strategies

Least-squares (discussed in [Chapters 1 and 4](#)) minimizes the sum of the squares of the residuals between a given fitting model and data. Linear least-squares, where the residuals are linear in the unknowns, has a closed-form solution which can be computed by taking the derivative of the residual with respect to each unknown and setting it to zero. It is commonly used in the engineering and applied sciences for fitting polynomial functions. Nonlinear least-squares typically requires iterative refinement based upon approximating the nonlinear least-squares with a linear least-squares at each iteration.

Gradient descent (discussed in [Chapters 4 and 6](#)) is the industry-leading, convex optimization method for high-dimensional systems. It minimizes residuals by computing the gradient of a given fitting function. The iterative procedure updates the solution by *moving downhill* in the residual space. The Newton–Raphson method is a one-dimensional version of gradient descent. Since it is often applied in high-dimensional settings, it is prone to find only local minima. Critical innovations for big data applications include stochastic gradient descent and the backpropagation algorithm, which makes the optimization amenable to computing the gradient itself.

Alternating descent method (ADM) (discussed in [Chapter 4](#)) avoids computations of the gradient by optimizing in one unknown at a time. Thus all unknowns are held constant while a line search (non-convex optimization) can be performed in a single variable. This variable is then updated and held constant while another of the unknowns is updated. The iterative procedure continues through all unknowns and the iteration procedure is repeated until a desired level of accuracy is achieved.

Augmented Lagrange method (ALM) (discussed in [Chapters 3 and 8](#)) is a class of algorithms for solving constrained optimization problems. They are similar to penalty methods in that they replace a constrained optimization problem by a series of unconstrained problems and add a penalty term to the objective which helps enforce the desired constraint. ALM adds another term designed to mimic a Lagrange multiplier. The augmented Lagrangian is not the same as the method of Lagrange multipliers.

Linear program and simplex method are the workhorse algorithms for convex optimization. A linear program has an objective function which is linear in the unknown, and the constraints consist of linear inequalities and equalities. By computing its feasible region, which is a convex polytope, the linear programming algorithm finds a point in the polyhedron where this function has the smallest (or largest) value if such a point exists. The simplex method is a specific iterative technique for linear programs which aims to take

a given basic feasible solution to another basic feasible solution for which the objective function is smaller, thus producing an iterative procedure for optimizing.

Most Common Equations and Symbols

Linear Algebra

Linear System of Equations

$$\mathbf{Ax} = \mathbf{b}.$$

The matrix $\mathbf{A} \in \mathbb{R}^{p \times n}$ and vector $\mathbf{b} \in \mathbb{R}^p$ are generally known, and the vector $\mathbf{x} \in \mathbb{R}^n$ is unknown.

Eigenvalue Equation

$$\mathbf{AT} = \mathbf{T}\mathbf{\Lambda}.$$

The columns ξ_k of the matrix $\mathbf{T}$ are the eigenvectors of $\mathbf{A} \in \mathbb{C}^{n \times n}$ corresponding to the eigenvalue λ_k : $\mathbf{A}\xi_k = \lambda_k\xi_k$. The matrix $\mathbf{\Lambda}$ is a diagonal matrix containing these eigenvalues, in the simple case with n distinct eigenvalues.

Change of Coordinates

$$\mathbf{x} = \mathbf{\Psi}\mathbf{a}.$$

The vector $\mathbf{x} \in \mathbb{R}^n$ may be written as $\mathbf{a} \in \mathbb{R}^n$ in the coordinate system given by the columns of $\mathbf{\Psi} \in \mathbb{R}^{n \times n}$.

Measurement Equation

$$\mathbf{y} = \mathbf{Cx}.$$

The vector $\mathbf{y} \in \mathbb{R}^p$ is a measurement of the state $\mathbf{x} \in \mathbb{R}^n$ by the measurement matrix $\mathbf{C} \in \mathbb{R}^{p \times n}$.

Singular Value Decomposition

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^* \approx \tilde{\mathbf{U}}\tilde{\mathbf{\Sigma}}\tilde{\mathbf{V}}^*.$$

The matrix $\mathbf{X} \in \mathbb{C}^{n \times m}$ may be decomposed into the product of three matrices $\mathbf{U} \in \mathbb{C}^{n \times n}$, $\mathbf{\Sigma} \in \mathbb{C}^{n \times m}$, and $\mathbf{V} \in \mathbb{C}^{m \times m}$. The matrices $\mathbf{U}$ and $\mathbf{V}$ are *unitary*, so that $\mathbf{U}\mathbf{U}^* = \mathbf{U}^*\mathbf{U} = \mathbf{I}_{n \times n}$ and $\mathbf{V}\mathbf{V}^* = \mathbf{V}^*\mathbf{V} = \mathbf{I}_{m \times m}$, where $*$ denotes complex conjugate transpose. The columns of $\mathbf{U}$ (respectively $\mathbf{V}$) are orthogonal, called left (respectively right) *singular vectors*. The matrix $\mathbf{\Sigma}$ contains decreasing, non-negative diagonal entries called *singular values*.

Often, $\mathbf{X}$ is approximated with a low-rank matrix $\tilde{\mathbf{X}} = \tilde{\mathbf{U}}\tilde{\mathbf{\Sigma}}\tilde{\mathbf{V}}^*$, where $\tilde{\mathbf{U}}$ and $\tilde{\mathbf{V}}$ contain the first $r \ll n$ columns of $\mathbf{U}$ and $\mathbf{V}$, respectively, and $\tilde{\mathbf{\Sigma}}$ contains the first $r \times r$ block of $\mathbf{\Sigma}$. The matrix $\tilde{\mathbf{U}}$ is often denoted $\mathbf{\Psi}$ in the context of spatial modes, reduced-order models, and sensor placement.

Regression and Optimization

Over-Determined and Under-Determined Optimization for Linear Systems

$$\begin{aligned} & \underset{\mathbf{x}}{\operatorname{argmin}}(\|\mathbf{Ax} - \mathbf{b}\|_2 + \lambda g(\mathbf{x})) && \text{or} \\ & \underset{\mathbf{x}}{\operatorname{argmin}} g(\mathbf{x}) && \text{subject to } \|\mathbf{Ax} - \mathbf{b}\|_2 \leq \epsilon. \end{aligned}$$

Here $g(\mathbf{x})$ is a regression penalty (with penalty parameter λ for over-determined systems). For over- and under-determined linear systems of equations, which result in either no solutions or an infinite number of solutions of $\mathbf{Ax} = \mathbf{b}$, a choice of constraint or penalty, which is also known as *regularization*, must be made in order to produce a solution.

Over-Determined and Under-Determined Optimization for Nonlinear Systems

$$\begin{aligned} & \underset{\mathbf{x}}{\operatorname{argmin}}(f(\mathbf{A}, \mathbf{x}, \mathbf{b}) + \lambda g(\mathbf{x})) && \text{or} \\ & \underset{\mathbf{x}}{\operatorname{argmin}} g(\mathbf{x}) && \text{subject to } f(\mathbf{A}, \mathbf{x}, \mathbf{b}) \leq \epsilon. \end{aligned}$$

This generalizes the linear system to a nonlinear system $f(\cdot)$ with regularization $g(\cdot)$. These over- and under-determined systems are often solved using gradient descent algorithms.

Compositional Optimization for Neural Networks

$$\underset{\mathbf{A}_j}{\operatorname{argmin}}(f_M(\mathbf{A}_M, \dots f_2(\mathbf{A}_2, (f_1(\mathbf{A}_1, \mathbf{x})) \dots)) + \lambda g(\mathbf{A}_j)).$$

Each $\mathbf{A}_k$ denotes the weight connecting the neural network from the k th to the $(k + 1)$ th layer. It is typically a massively under-determined system which is regularized by $g(\mathbf{A}_j)$. Composition and regularization are critical for generating expressive representations and preventing overfitting. The full network is often denoted $\mathbf{f}_\theta$.

Dynamical Systems and Reduced-Order Models

Nonlinear Ordinary Differential Equation (Dynamical System)

$$\frac{d}{dt}\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t; \boldsymbol{\beta}).$$

The vector $\mathbf{x}(t) \in \mathbb{R}^n$ is the state of the system evolving in time t , $\boldsymbol{\beta}$ are parameters, and $\mathbf{f}$ is the vector field. Generally, $\mathbf{f}$ is Lipschitz continuous to guarantee existence and uniqueness of solutions.

Linear Input–Output System

$$\begin{aligned} \frac{d}{dt}\mathbf{x} &= \mathbf{Ax} + \mathbf{Bu}, \\ \mathbf{y} &= \mathbf{Cx} + \mathbf{Du}. \end{aligned}$$

The state of the system is $\mathbf{x} \in \mathbb{R}^n$, the inputs (actuators) are $\mathbf{u} \in \mathbb{R}^q$, and the outputs (sensors) are $\mathbf{y} \in \mathbb{R}^p$. The matrices $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$, and $\mathbf{D}$ define the dynamics, the effect of actuation, the sensing strategy, and the effect of actuation feed-through, respectively.

Nonlinear Map (Discrete-Time Dynamical System)

$$\mathbf{x}_{k+1} = \mathbf{F}(\mathbf{x}_k).$$

The state of the system at the k th iteration is $\mathbf{x}_k \in \mathbb{R}^n$, and $\mathbf{F}$ is a possibly nonlinear mapping. Often, this map defines an iteration forward in time, so that $\mathbf{x}_k = \mathbf{x}(k\Delta t)$; in this case the flow map is denoted $\mathbf{F}_{\Delta t}$.

Koopman Operator Equation (Discrete-Time)

$$\mathcal{K}_t g = g \circ \mathbf{F}_t \implies \mathcal{K}_t \varphi = \lambda \varphi.$$

The linear Koopman operator $\mathcal{K}_t$ advances measurement functions of the state $g(\mathbf{x})$ with the flow $\mathbf{F}_t$. The eigenvalues and eigenvectors of $\mathcal{K}_t$ are λ and $\varphi(\mathbf{x})$, respectively. The operator $\mathcal{K}_t$ operates on a Hilbert space of measurements.

Nonlinear Partial Differential Equation (PDE)

$$\mathbf{u}_t = \mathbf{N}(\mathbf{u}, \mathbf{u}_x, \mathbf{u}_{xx}, \dots, x, t; \boldsymbol{\beta}).$$

The state of the PDE is $\mathbf{u}$, the nonlinear evolution operator is $\mathbf{N}$, subscripts denote partial differentiation, and x and t are the spatial and temporal variables, respectively. The PDE is parameterized by values in $\boldsymbol{\beta}$. The state $\mathbf{u}$ of the PDE may be a continuous function $u(x, t)$, or it may be discretized at several spatial locations, $\mathbf{u}(t) = [u(x_1, t) \ u(x_2, t) \ \dots \ u(x_n, t)]^T \in \mathbb{R}^n$.

Galerkin Expansion

The continuous Galerkin expansion is

$$u(x, t) \approx \sum_{k=1}^r a_k(t) \psi_k(x).$$

The functions $a_k(t)$ are temporal coefficients that capture the time dynamics, and $\psi_k(x)$ are spatial modes. For a high-dimensional discretized state, the Galerkin expansion becomes $\mathbf{u}(t) \approx \sum_{k=1}^r a_k(t) \boldsymbol{\psi}_k$. The spatial modes $\boldsymbol{\psi}_k \in \mathbb{R}^n$ may be the columns of $\boldsymbol{\Psi} = \tilde{\mathbf{U}}$.

Complete Symbols**Dimensions**

- K Number of non-zero entries in a K -sparse vector $\mathbf{s}$
- m Number of data snapshots (i.e., columns of $\mathbf{X}$)
- n Dimension of the state, $\mathbf{x} \in \mathbb{R}^n$
- p Dimension of the measurement or output variable, $\mathbf{y} \in \mathbb{R}^p$
- q Dimension of the input variable, $\mathbf{u} \in \mathbb{R}^q$
- r Rank of truncated SVD, or other low-rank approximation

Scalars

- s Frequency in Laplace domain
- t Time
- δ Learning rate in gradient descent
- Δt Time-step
- x Spatial variable
- Δx Spatial step
- σ Singular value
- λ Eigenvalue
- λ Sparsity parameter for sparse optimization ([Section 7.3](#))
- λ Lagrange multiplier ([Sections 3.7](#), [8.4](#), and [12.4](#))
- τ Threshold

Vectors

- $\mathbf{a}$ Vector of mode amplitudes of $\mathbf{x}$ in basis Ψ , $\mathbf{a} \in \mathbb{R}^r$
- $\mathbf{a}$ Action of reinforcement learning agent ([Chapter 11](#))
- $\mathbf{b}$ Vector of measurements in linear system $\mathbf{Ax} = \mathbf{b}$
- $\mathbf{b}$ Vector of DMD mode amplitudes ([Section 7.2](#))
- $\mathbf{Q}$ Vector containing potential function for PDE-FIND
- $\mathbf{r}$ Residual error vector
- $\mathbf{s}$ Sparse vector, $\mathbf{s} \in \mathbb{R}^n$ ([Chapter 3](#))
- $\mathbf{s}$ State of the environment in reinforcement learning ([Chapter 11](#))
- $\mathbf{u}$ Control variable ([Chapters 8](#), [9](#), and [10](#))
- $\mathbf{u}$ PDE state vector ([Chapters 12](#) and [13](#))
- $\mathbf{w}$ Exogenous inputs
- $\mathbf{w}_d$ Disturbances to system
- $\mathbf{w}_n$ Measurement noise
- $\mathbf{w}_r$ Reference to track
- $\mathbf{x}$ State of a system, $\mathbf{x} \in \mathbb{R}^n$
- $\mathbf{x}_k$ Snapshot of data at time t_k
- $\mathbf{x}_j$ Data sample $j \in Z := \{1, 2, \dots, m\}$ ([Chapters 5](#) and [6](#))
- $\tilde{\mathbf{x}}$ Reduced state, $\tilde{\mathbf{x}} \in \mathbb{R}^r$, so that $\mathbf{x} \approx \tilde{\mathbf{U}}\tilde{\mathbf{x}}$
- $\hat{\mathbf{x}}$ Estimated state of a system
- $\mathbf{y}$ Vector of measurements, $\mathbf{y} \in \mathbb{R}^p$
- $\mathbf{y}_j$ Data label $j \in Z := \{1, 2, \dots, m\}$ ([Chapters 5](#) and [6](#))
- $\hat{\mathbf{y}}$ Estimated output measurement
- $\mathbf{z}$ Transformed state, $\mathbf{x} = \mathbf{Tz}$ ([Chapters 8](#) and [9](#))
- ϵ Error vector
- β Bifurcation parameters
- ξ Eigenvector of Koopman operator ([Sections 7.4](#) and [7.5](#))
- ξ Sparse vector of coefficients ([Section 7.3](#))
- θ Neural network parameters
- ϕ DMD mode
- ψ POD mode
- Υ Vector of PDE measurements for PDE-FIND

Matrices

$\mathbf{A}$	Matrix for system of equations or dynamics
$\tilde{\mathbf{A}}$	Reduced dynamics on r -dimensional POD subspace
$\mathbf{A}_X$	Matrix representation of linear dynamics on the state $\mathbf{x}$
$\mathbf{A}_Y$	Matrix representation of linear dynamics on the observables $\mathbf{y}$
$(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D})$	Matrices for continuous-time state-space system
$(\mathbf{A}_d, \mathbf{B}_d, \mathbf{C}_d, \mathbf{D}_d)$	Matrices for discrete-time state-space system
$(\hat{\mathbf{A}}, \hat{\mathbf{B}}, \hat{\mathbf{C}}, \hat{\mathbf{D}})$	Matrices for state-space system in new coordinates $\mathbf{z} = \mathbf{T}^{-1}\mathbf{x}$
$(\tilde{\mathbf{A}}, \tilde{\mathbf{B}}, \tilde{\mathbf{C}}, \tilde{\mathbf{D}})$	Matrices for reduced state-space system with rank r
$\mathbf{B}$	Actuation input matrix
$\mathbf{C}$	Linear measurement matrix from state to measurements
$\mathcal{C}$	Controllability matrix
$\mathcal{F}$	Discrete Fourier transform
$\mathbf{G}$	Matrix representation of linear dynamics on the states and inputs $[\mathbf{x}^T \mathbf{u}^T]^T$
$\mathbf{H}$	Hankel matrix
$\mathbf{H}'$	Time-shifted Hankel matrix
$\mathbf{I}$	Identity matrix
$\mathbf{K}$	Matrix form of Koopman operator (Chapter 7)
$\mathbf{K}$	Closed-loop control gain (Chapter 8)
$\mathbf{K}_f$	Kalman filter estimator gain
$\mathbf{K}_r$	LQR control gain
$\mathbf{L}$	Low-rank portion of matrix $\mathbf{X}$ (Chapter 3)
$\mathcal{O}$	Observability matrix
$\mathbf{P}$	Unitary matrix that acts on columns of $\mathbf{X}$
$\mathbf{Q}$	Weight matrix for state penalty in LQR (Section 8.4)
$\mathbf{Q}$	Orthogonal matrix from QR factorization
$\mathbf{R}$	Weight matrix for actuation penalty in LQR (Section 8.4)
$\mathbf{R}$	Upper triangular matrix from QR factorization
$\mathbf{S}$	Sparse portion of matrix $\mathbf{X}$ (Chapter 3)
$\mathbf{T}$	Matrix of eigenvectors (Chapter 8)
$\mathbf{T}$	Change of coordinates (Chapters 8 and 9)
$\mathbf{U}$	Left singular vectors of $\mathbf{X}$, $\mathbf{U} \in \mathbb{R}^{n \times n}$
$\hat{\mathbf{U}}$	Left singular vectors of economy SVD of $\mathbf{X}$, $\mathbf{U} \in \mathbb{R}^{n \times m}$
$\tilde{\mathbf{U}}$	Left singular vectors (POD modes) of truncated SVD of $\mathbf{X}$, $\mathbf{U} \in \mathbb{R}^{n \times r}$
$\mathbf{V}$	Right singular vectors of $\mathbf{X}$, $\mathbf{V} \in \mathbb{R}^{m \times m}$
$\tilde{\mathbf{V}}$	Right singular vectors of truncated SVD of $\mathbf{X}$, $\mathbf{V} \in \mathbb{R}^{m \times r}$
Σ	Matrix of singular values of $\mathbf{X}$, $\Sigma \in \mathbb{R}^{n \times m}$
$\hat{\Sigma}$	Matrix of singular values of economy SVD of $\mathbf{X}$, $\Sigma \in \mathbb{R}^{m \times m}$
$\tilde{\Sigma}$	Matrix of singular values of truncated SVD of $\mathbf{X}$, $\Sigma \in \mathbb{R}^{r \times r}$
$\mathbf{W}$	Eigenvectors of $\tilde{\mathbf{A}}$
$\mathbf{W}_c$	Controllability Gramian
$\mathbf{W}_o$	Observability Gramian
$\mathbf{X}$	Data matrix, $\mathbf{X} \in \mathbb{R}^{n \times m}$
$\mathbf{X}'$	Time-shifted data matrix, $\mathbf{X}' \in \mathbb{R}^{n \times m}$
$\mathbf{Y}$	Projection of $\mathbf{X}$ matrix onto orthogonal basis in randomized SVD (Section 1.8)

$\mathbf{Y}$	Data matrix of observables, $\mathbf{Y} = \mathbf{g}(\mathbf{X})$, $\mathbf{Y} \in \mathbb{R}^{p \times m}$ (Chapter 7)
$\mathbf{Y}'$	Shifted data matrix of observables, $\mathbf{Y}' = \mathbf{g}(\mathbf{X}')$, $\mathbf{Y}' \in \mathbb{R}^{p \times m}$ (Chapter 7)
$\mathbf{Z}$	Sketch matrix for randomized SVD, $\mathbf{Z} \in \mathbb{R}^{n \times r}$ (Section 1.8)
Θ	Measurement matrix times sparsifying basis, $\Theta = \mathbf{C}\Psi$ (Chapter 3)
Θ	Matrix of candidate functions for SINDy (Section 7.3)
Γ	Matrix of derivatives of candidate functions for SINDy (Section 7.3)
Ξ	Matrix of coefficients of candidate functions for SINDy (Section 7.3)
Ξ	Matrix of nonlinear snapshots for DEIM (Section 13.5)
Λ	Diagonal matrix of eigenvalues
Υ	Input snapshot matrix, $\Upsilon \in \mathbb{R}^{q \times m}$
Φ	Matrix of DMD modes, $\Phi \triangleq \mathbf{X}'\mathbf{V}\Sigma^{-1}\mathbf{W}$
Ψ	Orthonormal basis (e.g., Fourier or POD modes)

Tensors

$(\mathcal{A}, \mathcal{B}, \mathcal{M})$ N -way array tensors of size $I_1 \times I_2 \times \cdots \times I_N$

Norms

$\ \cdot\ _0$	ℓ_0 pseudo-norm of a vector $\mathbf{x}$; the number of non-zero elements in $\mathbf{x}$
$\ \cdot\ _1$	ℓ_1 -norm of a vector $\mathbf{x}$ given by $\ \mathbf{x}\ _1 = \sum_{i=1}^n x_i $
$\ \cdot\ _2$	ℓ_2 -norm of a vector $\mathbf{x}$ given by $\ \mathbf{x}\ _2 = \sqrt{\sum_{i=1}^n (x_i^2)}$
$\ \cdot\ _2$	2-norm of a matrix $\mathbf{X}$ given by $\ \mathbf{X}\ _2 = \max_{\mathbf{v} \neq \mathbf{0}} \ \mathbf{X}\mathbf{v}\ _2 / \ \mathbf{v}\ _2$
$\ \cdot\ _F$	Frobenius norm of a matrix $\mathbf{X}$ given by $\ \mathbf{X}\ _F = \sqrt{\sum_{i=1}^n \sum_{j=1}^m X_{ij} ^2}$
$\ \cdot\ _*$	Nuclear norm of a matrix $\mathbf{X}$ given by $\ \mathbf{X}\ _* = \text{trace}(\sqrt{\mathbf{X}^*\mathbf{X}}) = \sum_{i=1}^m \sigma_i$ (for $m \leq n$)
$\langle \cdot, \cdot \rangle$	Inner product; for functions, $\langle f(x), g(x) \rangle = \int_{-\infty}^{\infty} f(x)g^*(x) dx$
$\langle \cdot, \cdot \rangle$	Inner product; for vectors, $\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^*\mathbf{v}$

Operators, Functions, and Maps

$\mathcal{F}$	Fourier transform
$\mathbf{F}$	Discrete-time dynamical system map
$\mathbf{F}_t$	Discrete-time flow map of dynamical system through time t
$\mathbf{f}_\theta$	Neural network (Chapter 6)
$\mathbf{f}$	Continuous-time dynamical system (Chapter 7)
$\mathcal{G}$	Gabor transform
$\mathbf{G}$	Transfer function from inputs to outputs (Chapter 8)
g	Scalar measurement function on $\mathbf{x}$
$\mathbf{g}$	Vector-valued measurement functions on $\mathbf{x}$
J	Cost function for control
ℓ	Loss function for support vector machines (Chapter 5)
$\mathcal{K}$	Koopman operator (continuous-time)
$\mathcal{K}_t$	Koopman operator associated with time- t flow map
$\mathcal{L}$	Laplace transform
$\mathbf{L}$	Loop transfer function (Chapter 8)

L	Linear partial differential equation (Chapters 12 and 13)
N	Nonlinear partial differential equation
$\mathcal{O}$	Order of magnitude
Q	Quality function (Chapter 11)
$\Re$	Real part
S	Sensitivity function (Chapter 8)
T	Complementary sensitivity function (Chapter 8)
V	Value function (Chapter 11)
$\mathcal{W}$	Wavelet transform
μ	Incoherence between measurement matrix $\mathbf{C}$ and basis Ψ
κ	Condition number
π	Policy function for agent in reinforcement learning (Chapter 11)
φ	Koopman eigenfunction
∇	Gradient operator
$*$	Convolution operator

Most Common Acronyms

CNN	Convolutional neural network
DL	Deep learning
DMD	Dynamic mode decomposition
FFT	Fast Fourier transform
ODE	Ordinary differential equation
PCA	Principal component analysis
PDE	Partial differential equation
POD	Proper orthogonal decomposition
RL	Reinforcement learning
ROM	Reduced-order model
SVD	Singular value decomposition

Other Acronyms

ADM	Alternating directions method
AIC	Akaike information criterion
ALM	Augmented Lagrange multiplier
ANN	Artificial neural network
ARMA	Autoregressive moving average
ARMAX	Autoregressive moving average with exogenous input
BIC	Bayesian information criterion
BPOD	Balanced proper orthogonal decomposition
DMDc	Dynamic mode decomposition with control
CCA	Canonical correlation analysis
CFD	Computational fluid dynamics
CoSaMP	Compressive sampling matching pursuit

CWT	Continuous wavelet transform
DEIM	Discrete empirical interpolation method
DCT	Discrete cosine transform
DFT	Discrete Fourier transform
DMDc	Dynamic mode decomposition with control
DNS	Direct numerical simulation
DP	Dynamic programming
DQN	Deep Q network
DRL	Deep reinforcement learning
DWT	Discrete wavelet transform
ECOG	Electrocorticography
eDMD	Extended DMD
EIM	Empirical interpolation method
EM	Expectation maximization
EOF	Empirical orthogonal functions
ERA	Eigensystem realization algorithm
ESC	Extremum-seeking control
GMM	Gaussian mixture model
HAVOK	Hankel alternative view of Koopman
HER	Hindsight experience replay
HJB	Hamilton–Jacobi–Bellman equation
JL	Johnson–Lindenstrauss
KL	Kullback–Leibler
ICA	Independent component analysis
KLT	Karhunen–Loève transform
LAD	Least absolute deviations
LASSO	Least absolute shrinkage and selection operator
LDA	Linear discriminant analysis
LQE	Linear–quadratic estimator
LQG	Linear–quadratic Gaussian controller
LQR	Linear–quadratic regulator
LTi	Linear time-invariant system
MDP	Markov decision process
MIMO	Multiple-input, multiple-output
MLC	Machine learning control
MPE	Missing point estimation
mrDMD	Multi-resolution dynamic mode decomposition
NARMAX	Nonlinear autoregressive model with exogenous inputs
NLS	Nonlinear Schrödinger equation
OKID	Observer Kalman filter identification
PBH	Popov–Belevitch–Hautus test
PCP	Principal component pursuit
PDE-FIND	Partial differential equation functional identification of nonlinear dynamics
PDF	Probability density function
PID	Proportional–integral–derivative control

PINN	Physics-informed neural network
PIV	Particle image velocimetry
RIP	Restricted isometry property
rSVD	Randomized SVD
RKHS	Reproducing kernel Hilbert space
RNN	Recurrent neural network
RPCA	Robust principal component analysis
SGD	Stochastic gradient descent
SINDy	Sparse identification of nonlinear dynamics
SINDYc	SINDy with control
SISO	Single-input, single-output
SRC	Sparse representation for classification
SSA	Singular spectrum analysis
STFT	Short-time Fourier transform
STLS	Sequential thresholded least-squares
SVM	Support vector machine
TICA	Time-lagged independent component analysis
VAC	Variational approach of conformation dynamics

Part I

Dimensionality Reduction and Transforms

1 Singular Value Decomposition (SVD)

The singular value decomposition (SVD) is among the most important matrix factorizations of the computational era, providing a foundation for nearly all of the data methods in this book. The SVD provides a numerically stable matrix decomposition that can be used for a variety of purposes and is guaranteed to exist. We will use the SVD to obtain optimal low-rank approximations to matrices and to perform pseudo-inverses of non-square matrices to find a solution to the system of equations $\mathbf{Ax} = \mathbf{b}$. The SVD will also be used as the underlying algorithm of principal component analysis (PCA), where high-dimensional data is decomposed into its most statistically descriptive factors. SVD/PCA has been applied to a wide variety of problems in science and engineering.

In a sense, the SVD generalizes the concept of the fast Fourier transform (FFT), which will be the subject of the next chapter. Many engineering texts begin with the FFT, as it is the basis of many classical analytical and numerical results. However, the FFT works in idealized settings, and the SVD is a more generic data-driven technique. Because this book is focused on data, we begin with the SVD, which may be thought of as providing a basis that is *tailored* to the specific data, as opposed to the FFT, which provides a *generic* basis.

In many domains, complex systems will generate data that is naturally arranged in large matrices, or more generally in arrays. For example, a time series of data from an experiment or a simulation may be arranged in a matrix, with each column containing all of the measurements at a given time. If the data at each instant in time is multi-dimensional, as in a high-resolution simulation of the weather in three spatial dimensions, it is possible to reshape or *flatten* this data into a high-dimensional column vector, forming the columns of a large matrix. Similarly, the pixel values in a grayscale image may be stored in a matrix, or these images may be reshaped into large column vectors in a matrix to represent the frames of a movie. Remarkably, the data generated by these systems is typically low-rank, meaning that there are a few dominant patterns that explain the high-dimensional data. The SVD is a numerically robust and efficient method of extracting these patterns from data.

1.1 Overview

Here we introduce the singular value decomposition (SVD) and develop an intuition for how to apply the SVD by demonstrating its use on a number of motivating examples. The SVD will provide a foundation for many other techniques developed in this book, including classification methods in [Chapter 5](#), the dynamic mode decomposition (DMD) in [Chapter 7](#), and the proper orthogonal decomposition (POD) in [Chapter 12](#). Detailed mathematical properties are discussed in the following sections.

High dimensionality is a common challenge in processing data from complex systems. These systems may involve large measured data sets including audio, image, or video data. The data may also be generated from a physical system, such as neural recordings from a brain, or fluid velocity measurements from a simulation or experiment. In many naturally occurring systems, it is observed that data exhibit dominant patterns, which may be characterized by a low-dimensional attractor or manifold [333, 334].

As an example, consider images, which typically contain a large number of measurements (pixels), and are therefore elements of a high-dimensional vector space. However, most images are highly compressible, meaning that the relevant information may be represented in a much lower-dimensional subspace. The compressibility of images will be discussed in depth throughout this book. Complex fluid systems, such as the Earth's atmosphere or the turbulent wake behind a vehicle, also provide compelling examples of the low-dimensional structure underlying a high-dimensional state space. Although high-fidelity fluid simulations typically require at least millions or billions of degrees of freedom, there are often dominant coherent structures in the flow, such as periodic vortex shedding behind vehicles or hurricanes in the weather.

The SVD provides a systematic way to determine a low-dimensional approximation to high-dimensional data in terms of dominant patterns. This technique is *data-driven* in that patterns are discovered purely from data, without the addition of expert knowledge or intuition. The SVD is numerically stable and provides a hierarchical representation of the data in terms of a new coordinate system defined by dominant correlations within the data. Moreover, the SVD is guaranteed to exist for any matrix, unlike the eigendecomposition.

The SVD has many powerful applications beyond dimensionality reduction of high-dimensional data. It is used to compute the pseudo-inverse of non-square matrices, providing solutions to under-determined or over-determined matrix equations, $\mathbf{Ax} = \mathbf{b}$. We will also use the SVD to de-noise data sets. The SVD is likewise important to characterize the input and output geometry of a linear map between vector spaces. These applications will all be explored in this chapter, providing an intuition for matrices and high-dimensional data.

Definition of the SVD

Generally, we are interested in analyzing a large data set $\mathbf{X} \in \mathbb{C}^{n \times m}$:

$$\mathbf{X} = \begin{bmatrix} | & | & \cdots & | \\ \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_m \\ | & | & \cdots & | \end{bmatrix}. \quad (1.1)$$

The columns $\mathbf{x}_k \in \mathbb{C}^n$ may be measurements from simulations or experiments. For example, columns may represent images that have been reshaped into column vectors with as many elements as pixels in the image. The column vectors may also represent the state of a physical system that is evolving in time, such as the fluid velocity at a set of discrete points, a set of neural measurements, or the state of a weather simulation with one square kilometer resolution.

The index k is a label indicating the k th distinct set of measurements. For many of the examples in this book, $\mathbf{X}$ will consist of a *time series* of data, and $\mathbf{x}_k = \mathbf{x}(k\Delta t)$. Often the *state dimension* n is very large, on the order of millions or billions of degrees of freedom.

The columns are often called *snapshots*, and m is the number of snapshots in $\mathbf{X}$. For many systems $n \gg m$, resulting in a *tall-skinny* matrix, as opposed to a *short-fat* matrix when $n \ll m$.

The SVD is a unique matrix decomposition that exists for every complex-valued matrix $\mathbf{X} \in \mathbb{C}^{n \times m}$:

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*, \quad (1.2)$$

where $\mathbf{U} \in \mathbb{C}^{n \times n}$ and $\mathbf{V} \in \mathbb{C}^{m \times m}$ are *unitary* matrices¹ with orthonormal columns, and $\mathbf{\Sigma} \in \mathbb{R}^{n \times m}$ is a matrix with real, non-negative entries on the diagonal and zeros off the diagonal. Here $*$ denotes the complex conjugate transpose.² As we will discover throughout this chapter, the condition that $\mathbf{U}$ and $\mathbf{V}$ are unitary is used extensively.

When $n \geq m$, the matrix $\mathbf{\Sigma}$ has at most m non-zero elements on the diagonal, and may be written as

$$\mathbf{\Sigma} = \begin{bmatrix} \hat{\mathbf{\Sigma}} \\ \mathbf{0} \end{bmatrix}.$$

Therefore, it is possible to *exactly* represent $\mathbf{X}$ using the *economy* SVD:

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^* = \begin{bmatrix} \hat{\mathbf{U}} & \hat{\mathbf{U}}^\perp \end{bmatrix} \begin{bmatrix} \hat{\mathbf{\Sigma}} \\ \mathbf{0} \end{bmatrix} \mathbf{V}^* = \hat{\mathbf{U}}\hat{\mathbf{\Sigma}}\mathbf{V}^*. \quad (1.3)$$

The full SVD and economy SVD are shown in Fig. 1.1. The columns of $\hat{\mathbf{U}}^\perp$ span a vector space that is complementary and orthogonal to that spanned by $\hat{\mathbf{U}}$. The columns of $\mathbf{U}$ are called *left singular vectors* of $\mathbf{X}$ and the columns of $\mathbf{V}$ are *right singular vectors*. The diagonal elements of $\hat{\mathbf{\Sigma}} \in \mathbb{C}^{m \times m}$ are called *singular values* and they are ordered from largest to smallest. The rank of $\mathbf{X}$ is equal to the number of non-zero singular values. We will show in Section 1.2 that the SVD can also be used to obtain an optimal rank- r approximation of $\mathbf{X}$ for $r < m$.

Computing the SVD

The SVD is a cornerstone of computational science and engineering, and the numerical implementation of the SVD is both important and mathematically enlightening. That said, most standard numerical implementations are mature and a simple interface exists in many modern computer languages, allowing us to abstract away the details underlying the SVD computation. For most purposes, we simply use the SVD as a part of a larger effort, and we take for granted the existence of efficient and stable numerical algorithms. Numerically, the SVD may be computed by first reducing the matrix $\mathbf{X}$ to a bidiagonal matrix and then using an iterative algorithm to compute the SVD of the bidiagonal matrix. For matrices with high aspect ratio (i.e., $n \gg m$), then the first step may be achieved by first computing a QR factorization to reduce $\mathbf{X}$ to an upper triangular matrix, followed by Householder reflections to reduce this upper triangular matrix into a bidiagonal form. The second step may be performed using a modified QR algorithm developed by Golub & Kahan [284]. Details of the QR factorization are beyond the scope of this book, although it is a straightforward greedy method that is closely related to Gram–Schmidt orthogonalization.

¹ A square matrix $\mathbf{U}$ is unitary if $\mathbf{U}\mathbf{U}^* = \mathbf{U}^*\mathbf{U} = \mathbf{I}$.

² For real-valued matrices, this is the same as the regular transpose $\mathbf{X}^* = \mathbf{X}^T$.

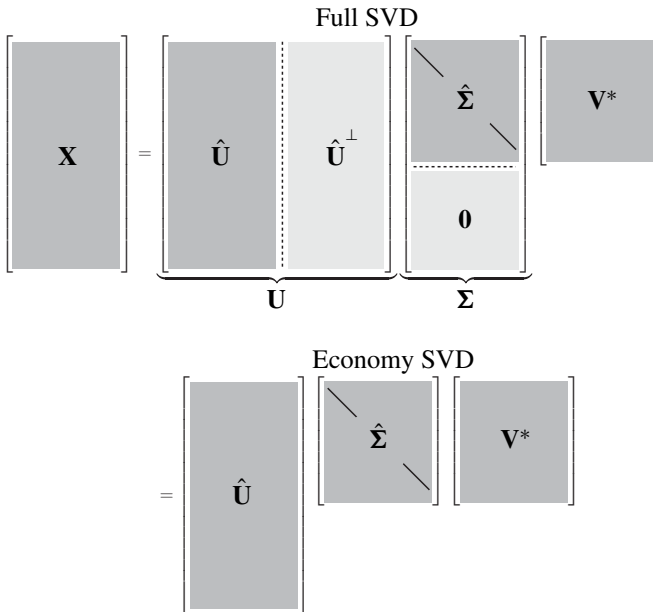


Figure 1.1 Schematic of matrices in the full and economy SVD.

There are numerous variations, alternatives, and important results on the computation of the SVD; for a more thorough discussion, see [143, 284, 285, 317, 388, 709]. Randomized numerical algorithms are increasingly used to compute the SVD of very large matrices as discussed in Section 1.8.

MATLAB

In MATLAB, computing the SVD is straightforward:

```
>> X = randn(5,3); % Create a 5x3 random data matrix
>> [U,S,V] = svd(X); % Singular value decomposition
```

For non-square matrices X , the economy SVD is more efficient:

```
>> [Uhat,Shat,V] = svd(X,'econ'); % Economy sized SVD
```

Python³

```
>>> import numpy as np
>>> X = np.random.rand(5, 3) # Create random data matrix
>>> U, s, VT = np.linalg.svd(X,full_matrices=True) #Full SVD
>>> Uhat, Shat, VThat = np.linalg.svd(X,full_matrices=False)
# Economy SVD
```

R

```
> X <- replicate(3, rnorm(5))
> s <- svd(X)
> U <- s$u
> S <- diag(s$d)
> V <- s$v
```

³ Note that Python outputs the transpose of V

Mathematica

```
In:= X=RandomReal[{0,1},{5,3}]
In:= {U,S,V} = SingularValueDecomposition[X]
```

Other Languages

The SVD is also available in other languages, such as Fortran and C++. In fact, most SVD implementations are based on the LAPACK (Linear Algebra Package) [19] in Fortran. The SVD routine is designated **DGESVD** in LAPACK, and this is wrapped in the C++ libraries **Armadillo** and **Eigen**.

Historical Perspective

The SVD has a long and rich history, ranging from early work developing the theoretical foundations to modern work on computational stability and efficiency. There is an excellent historical review by Stewart [674], which provides context and many important details. The review focuses on the early theoretical work of Beltrami and Jordan (1873), Sylvester (1889), Schmidt (1907), and Weyl (1912). It also discusses more recent work, including the seminal computational work of Golub and collaborators [284, 285]. In addition, there are many excellent chapters on the SVD in modern texts [24, 419, 709].

Uses in This Book and Assumptions of the Reader

The SVD is the basis for many related techniques in dimensionality reduction. These methods include principal component analysis (PCA) in statistics [338, 339, 550], the Karhunen–Loève transform (KLT) [373, 452], empirical orthogonal functions (EOFs) in climate [458], the proper orthogonal decomposition (POD) in fluid dynamics [334], and canonical correlation analysis (CCA) [175]. Although developed independently in a range of diverse fields, many of these methods only differ in how the data is collected and pre-processed. There is an excellent discussion about the relationship between the SVD, the KLT, and PCA by Gerbrands [274].

The SVD is also widely used in system identification and control theory to obtain reduced-order models that are balanced in the sense that states are hierarchically ordered in terms of their ability to be observed by measurements and controlled by actuation [508].

For this chapter, we assume that the reader is familiar with linear algebra, with some experience in computation and numerics. For review, there are a number of excellent books on numerical linear algebra, with discussions on the SVD [24, 419, 709].

1.2 Matrix Approximation

Perhaps the most useful and defining property of the SVD is that it provides an *optimal* low-rank approximation to a matrix $\mathbf{X}$. In fact, the SVD provides a *hierarchy* of low-rank approximations, since a rank- r approximation is obtained by keeping the leading r singular values and vectors, and discarding the rest.

Because $\mathbf{\Sigma}$ is diagonal, it is possible to express the matrix $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$ as a sum of rank-one matrices:

$$\mathbf{X} = \sum_{k=1}^m \sigma_k \mathbf{u}_k \mathbf{v}_k^* = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^* + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^* + \cdots + \sigma_m \mathbf{u}_m \mathbf{v}_m^*, \quad (1.4)$$

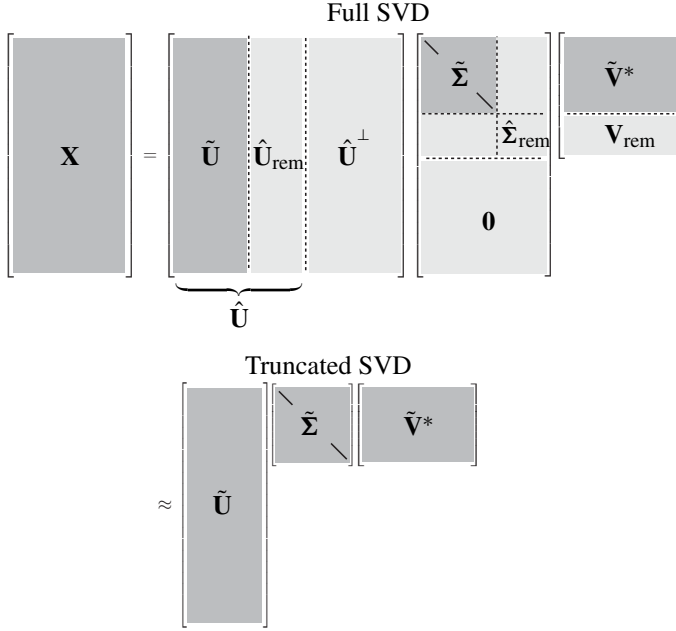


Figure 1.2 Schematic of truncated SVD. The subscript ‘rem’ denotes the remainder of $\hat{\mathbf{U}}$, $\hat{\Sigma}$, or $\hat{\mathbf{V}}$ after truncation.

where σ_k is the k th diagonal entry of Σ , and $\mathbf{u}_k$ and $\mathbf{v}_k$ are the k th columns of $\mathbf{U}$ and $\mathbf{V}$, respectively. This is known as the *dyadic* summation. The singular values σ_k are arranged in decreasing order, $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_m \geq 0$, so each subsequent rank-one matrix $\sigma_k \mathbf{u}_k \mathbf{v}_k^*$ is less important than the previous matrix in capturing the information in $\mathbf{X}$. For many systems, the singular values σ_k decrease rapidly, and it is possible to obtain a good approximation of $\mathbf{X}$ by truncating at some rank r :

$$\mathbf{X} \approx \tilde{\mathbf{X}} = \sum_{k=1}^r \sigma_k \mathbf{u}_k \mathbf{v}_k^* = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^* + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^* + \dots + \sigma_r \mathbf{u}_r \mathbf{v}_r^*. \quad (1.5)$$

Here, we establish the notation that a truncated SVD basis (and the resulting approximated matrix $\tilde{\mathbf{X}}$) will be denoted by $\tilde{\mathbf{X}} = \tilde{\mathbf{U}} \tilde{\Sigma} \tilde{\mathbf{V}}^*$, where $\tilde{\mathbf{U}}$ and $\tilde{\mathbf{V}}$ contain the first r columns of $\mathbf{U}$ and $\mathbf{V}$, and $\tilde{\Sigma}$ contains the first $r \times r$ sub-block of Σ . The truncated SVD is illustrated in Fig. 1.2, with $\tilde{\mathbf{U}}$, $\tilde{\Sigma}$, and $\tilde{\mathbf{V}}$ denoting the truncated matrices. If $\mathbf{X}$ does not have full rank, then some of the singular values in $\tilde{\Sigma}$ may be zero, and the truncated SVD may still be exact. However, for truncation values r that are smaller than the number of non-zero singular values (i.e., the rank of $\mathbf{X}$), the truncated SVD only approximates $\mathbf{X}$.

For a given rank r , there is no better approximation for $\mathbf{X}$, in the ℓ_2 sense, than the truncated SVD approximation $\tilde{\mathbf{X}}$. The Eckart–Young theorem below will state this precisely and provide expressions for the error of the truncated approximation. There are numerous choices for the truncation rank r , and they are discussed in Section 1.7. Thus, high-dimensional data may be well described by a few dominant patterns given by the columns of $\tilde{\mathbf{U}}$ and $\tilde{\mathbf{V}}$.

This is an important property of the SVD, and we will return to it many times. There are numerous examples of data sets that contain high-dimensional measurements, resulting

in a large data matrix $\mathbf{X}$. However, there are often dominant low-dimensional patterns in the data, and the truncated SVD basis $\tilde{\mathbf{U}}$ provides a coordinate transformation from the high-dimensional measurement space into a low-dimensional pattern space. This has the benefit of *reducing* the size and dimension of large data sets, yielding a tractable basis for visualization and analysis. Finally, many systems considered in this text are *dynamic* (see [Chapter 7](#)), and the SVD basis provides a hierarchy of modes that characterize the observed attractor, on which we may project a low-dimensional dynamical system to obtain reduced-order models (see [Chapter 13](#)).

Optimal Approximation and Error Bounds

Schmidt (of Gram–Schmidt) generalized the SVD to function spaces and developed an approximation theorem, establishing the truncated SVD $\tilde{\mathbf{X}}$ as the optimal low-rank approximation of the underlying matrix $\mathbf{X}$ [637]. Schmidt’s approximation theorem was rediscovered by Eckart and Young [227], and is sometimes referred to as the Eckart–Young theorem.

Theorem 1.1 (Eckart–Young [227]) *The optimal rank- r approximation to $\mathbf{X}$, in a least-squares sense, is given by the rank- r SVD truncation $\tilde{\mathbf{X}}$:*

$$\underset{\tilde{\mathbf{X}}, \text{ s.t. rank}(\tilde{\mathbf{X}})=r}{\operatorname{argmin}} \quad \|\mathbf{X} - \tilde{\mathbf{X}}\|_F = \tilde{\mathbf{U}}\tilde{\Sigma}\tilde{\mathbf{V}}^*. \quad (1.6)$$

Again, $\tilde{\mathbf{U}}$ and $\tilde{\mathbf{V}}$ denote the first r leading columns of $\mathbf{U}$ and $\mathbf{V}$, and $\tilde{\Sigma}$ contains the leading $r \times r$ sub-block of Σ . The Frobenius norm above is defined as $\|\mathbf{X}\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^m |X_{ij}|^2}$, which is equivalent to the 2-norm of the vectorized matrix $\mathbf{X}(:)$.

Thus, the Eckart–Young theorem guarantees that the truncated SVD provides the best matrix approximation of a given rank in the Frobenius norm. It is also possible to exactly quantify the error of the rank- r SVD approximation:

$$\|\mathbf{X} - \tilde{\mathbf{X}}\|_F^2 = \sum_{k=r+1}^m \sigma_k^2. \quad (1.7)$$

Thus, all other rank- r matrices $\tilde{\mathbf{X}}$ will have at least this much error. Because the error scales with the size and magnitude of $\mathbf{X}$, it is often more useful to consider the relative error

$$\frac{\|\mathbf{X} - \tilde{\mathbf{X}}\|_F^2}{\|\mathbf{X}\|_F^2}. \quad (1.8)$$

This expression for the relative error in the Frobenius norm has two intuitive interpretations. If the columns of $\mathbf{X}$ are velocity fields, for example from a discretized fluid flow simulation, then this error is related to the fraction of the *kinetic energy* that is missing in the approximation $\tilde{\mathbf{X}}$. More generally, the squared Frobenius norm error of mean-subtracted data has the interpretation of the amount of missing variance in the approximation $\tilde{\mathbf{X}}$. This statistical interpretation will be explored more in [Section 1.5](#).

Remarkably, the SVD also provides an optimal rank- r approximation in the matrix 2-norm, also known as the spectral norm:

$$\underset{\tilde{\mathbf{X}}, \text{ s.t. rank}(\tilde{\mathbf{X}})=r}{\operatorname{argmin}} \quad \|\mathbf{X} - \tilde{\mathbf{X}}\|_2 = \tilde{\mathbf{U}}\tilde{\Sigma}\tilde{\mathbf{V}}^*. \quad (1.9)$$

The 2-norm of a matrix $\mathbf{X}$ is induced by the vector 2-norm and is given by

$$\|\mathbf{X}\|_2 = \max_{\mathbf{v} \neq \mathbf{0}} \frac{\|\mathbf{X}\mathbf{v}\|_2}{\|\mathbf{v}\|_2}.$$

The error expression for the rank- r SVD approximation is even simpler in the 2-norm:

$$\|\mathbf{X} - \tilde{\mathbf{X}}\|_2 = \sigma_{r+1}. \quad (1.10)$$

This error expression is rather simple to derive by expanding

$$\mathbf{X} - \tilde{\mathbf{X}} = \sum_{k=r+1}^m \sigma_k \mathbf{u}_k \mathbf{v}_k^*. \quad (1.11)$$

Since each of the $\mathbf{v}_k$ vectors is orthonormal, the maximum $\|(\mathbf{X} - \tilde{\mathbf{X}})\mathbf{v}\|_2 = \sigma_{r+1}$ is achieved for $\mathbf{v} = \mathbf{v}_{r+1}$.

Example: Image Compression

We demonstrate the idea of matrix approximation with a simple example: image compression. A recurring theme throughout this book is that large data sets often contain underlying patterns that facilitate low-rank representations. Natural images present a simple and intuitive example of this inherent *compressibility*. A grayscale image may be thought of as a real-valued matrix $\mathbf{X} \in \mathbb{R}^{n \times m}$, where n and m are the number of pixels in the vertical and horizontal directions, respectively.³ Depending on the basis of representation (pixel space, Fourier frequency domain, SVD transform coordinates), images may have very compact approximations.

Consider the image of Mordecai the snow dog in Fig. 1.3. This image has 2000×1500 pixels. It is possible to take the SVD of this image and plot the diagonal singular values, as in Fig. 1.4. Figure 1.3 shows the approximate matrix $\tilde{\mathbf{X}}$ for various truncation values r . By $r = 100$, the reconstructed image is quite accurate, and the singular values account for almost 80% of the total cumulative sum of the singular values. The squared error is less than 4% in the Frobenius norm. The SVD truncation results in a compression of the original image, since only the first 100 columns of $\mathbf{U}$ and $\mathbf{V}$, along with the first 100 diagonal elements of Σ , must be stored in $\tilde{\mathbf{U}}$, $\tilde{\Sigma}$, and $\tilde{\mathbf{V}}$.

Code 1.1 [MATLAB] Use SVD to compress image.

```
% First, we load the image
A=imread(' ../DATA/dog.jpg');
X=double(rgb2gray(A)); % Convert RGB->gray, 256 bit->double.
nx = size(X,1); ny = size(X,2);
imagesc(X), axis off, colormap gray

% Take the SVD
[U,S,V] = svd(X);

% Approximate matrix with truncated SVD for various ranks r
for r=[5 20 100] % Truncation value
    Xapprox = U(:,1:r)*S(1:r,1:r)*V(:,1:r)'; % Approx. image
    figure, imagesc(Xapprox), axis off
    title(['r=', num2str(r, '%d')]);
end
```

³ It is not uncommon for image size to be specified as horizontal by vertical, i.e., $\mathbf{X}^T \in \mathbb{R}^{m \times n}$, although we stick with vertical by horizontal to be consistent with generic matrix notation.

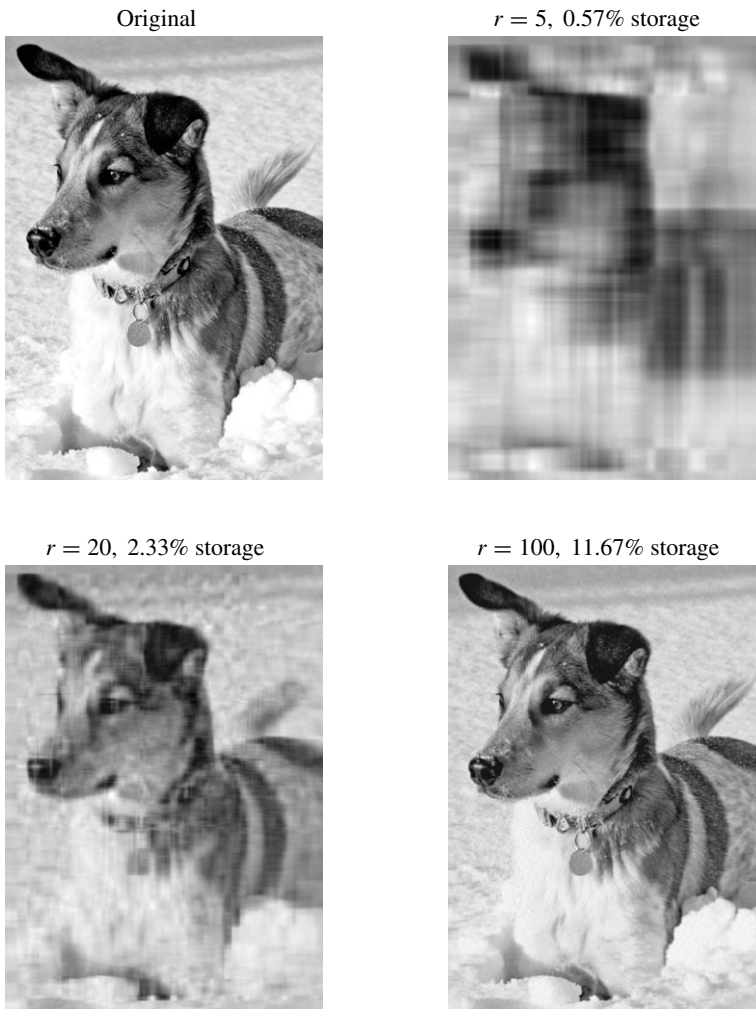


Figure 1.3 Image compression of Mordecai the snow dog, truncating the SVD at various ranks r . Original image resolution is 2000×1500 .

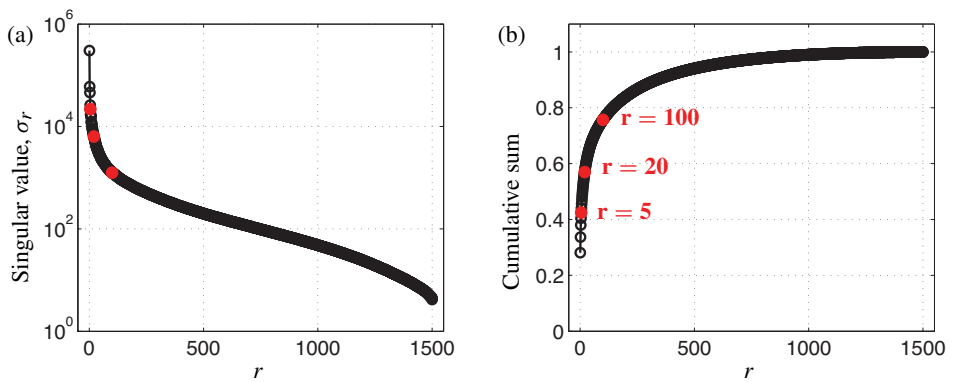


Figure 1.4 (a) Singular values σ_r and (b) cumulative sum $\sum_{k=1}^r \sigma_k$ of the first r singular values.

```
% Plot singular values and cumulative sum
subplot(1,2,1), semilogy(diag(S), 'k')
subplot(1,2,2), plot(cumsum(diag(S))/sum(diag(S)), 'k')
```

Code 1.1 [Python] Use SVD to compress image.

```
# First, we load the image
from matplotlib.image import imread
A = imread(os.path.join('..', 'DATA', 'dog.jpg'))
X = np.mean(A, -1); # Convert RGB to grayscale
img = plt.imshow(X)
# Take the SVD
U, S, VT = np.linalg.svd(X, full_matrices=False)
S = np.diag(S)
# Approximate matrix with truncated SVD for various ranks r
for r in (5, 20, 100): # Construct approximate image
    Xapprox = U[:, :r] @ S[0:r, :r] @ VT[:, r, :]
    img = plt.imshow(Xapprox)
    plt.show()
# Plot singular values and cumulative sum
plt.semilogy(np.diag(S))
plt.plot(np.cumsum(np.diag(S))/np.sum(np.diag(S)))
```

1.3

Mathematical Properties and Manipulations

Here we describe important mathematical properties of the SVD, including geometric interpretations of the unitary matrices $\mathbf{U}$ and $\mathbf{V}$, as well as a discussion of the SVD in terms of dominant correlations in the data $\mathbf{X}$. The relationship between the SVD and correlations in the data will be explored more in [Section 1.5](#) on principal component analysis.

Interpretation as Dominant Correlations

The SVD is closely related to an eigenvalue problem involving the correlation matrices $\mathbf{X}\mathbf{X}^*$ and $\mathbf{X}^*\mathbf{X}$, shown in [Fig. 1.5](#) for a specific image, and in [Figs. 1.6](#) and [1.7](#) for generic matrices. If we plug [\(1.3\)](#) into the row-wise correlation matrix $\mathbf{X}\mathbf{X}^*$ and the column-wise correlation matrix $\mathbf{X}^*\mathbf{X}$, we find

$$\mathbf{X}\mathbf{X}^* = \mathbf{U} \begin{bmatrix} \hat{\Sigma} \\ \mathbf{0} \end{bmatrix} \mathbf{V}^* \mathbf{V} \begin{bmatrix} \hat{\Sigma} & \mathbf{0} \end{bmatrix} \mathbf{U}^* = \mathbf{U} \begin{bmatrix} \hat{\Sigma}^2 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \mathbf{U}^*, \quad (1.12a)$$

$$\mathbf{X}^*\mathbf{X} = \mathbf{V} \begin{bmatrix} \hat{\Sigma} & \mathbf{0} \end{bmatrix} \mathbf{U}^* \mathbf{U} \begin{bmatrix} \hat{\Sigma} \\ \mathbf{0} \end{bmatrix} \mathbf{V}^* = \mathbf{V} \hat{\Sigma}^2 \mathbf{V}^*. \quad (1.12b)$$

Recalling that $\mathbf{U}$ and $\mathbf{V}$ are unitary, $\mathbf{U}$, $\hat{\Sigma}$, and $\mathbf{V}$ are solutions to the following eigenvalue problems:

$$\mathbf{X}\mathbf{X}^* \mathbf{U} = \mathbf{U} \begin{bmatrix} \hat{\Sigma}^2 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix}, \quad (1.13a)$$

$$\mathbf{X}^* \mathbf{X} \mathbf{V} = \mathbf{V} \hat{\Sigma}^2. \quad (1.13b)$$

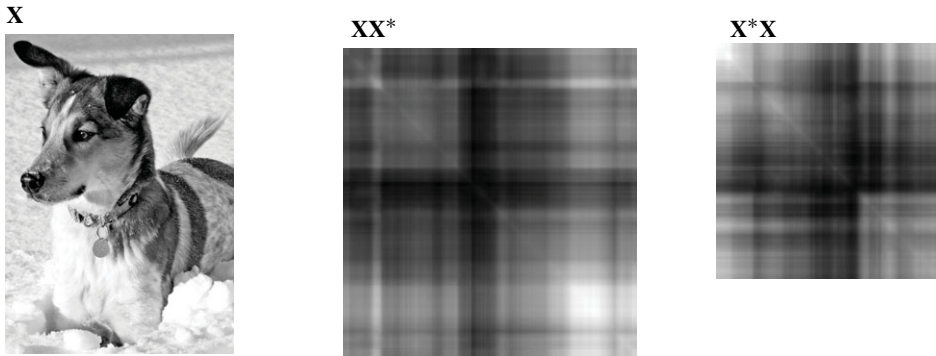


Figure 1.5 Correlation matrices $\mathbf{XX}^*$ and $\mathbf{X}^*\mathbf{X}$ for a matrix $\mathbf{X}$ obtained from an image of a dog. Note that both correlation matrices are symmetric.

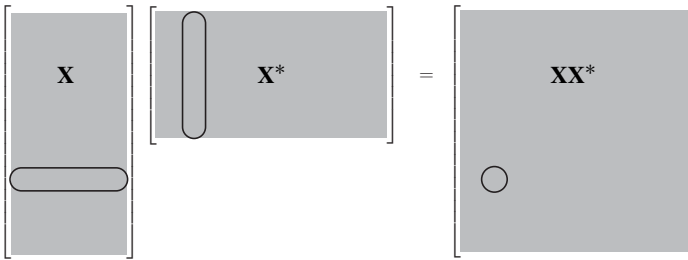


Figure 1.6 Correlation matrix $\mathbf{XX}^*$ is formed by taking the inner product of rows of $\mathbf{X}$.

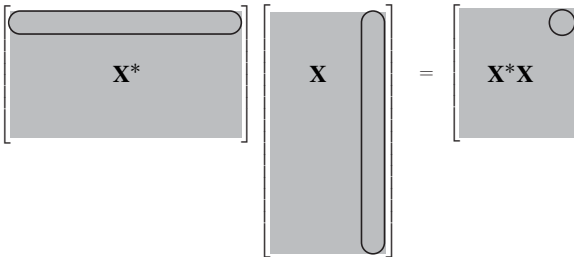


Figure 1.7 Correlation matrix $\mathbf{X}^*\mathbf{X}$ is formed by taking the inner product of columns of $\mathbf{X}$.

In other words, each non-zero singular value of $\mathbf{X}$ is a positive square root of an eigenvalue of $\mathbf{X}^*\mathbf{X}$ and of $\mathbf{XX}^*$, which have the same non-zero eigenvalues. It follows that, if $\mathbf{X}$ is self-adjoint (i.e., $\mathbf{X} = \mathbf{X}^*$), then the singular values of $\mathbf{X}$ are equal to the absolute value of the eigenvalues of $\mathbf{X}$.

This provides an intuitive interpretation of the SVD, where the columns of $\mathbf{U}$ are eigenvectors of the correlation matrix $\mathbf{XX}^*$, and the columns of $\mathbf{V}$ are eigenvectors of $\mathbf{X}^*\mathbf{X}$. We choose to arrange the singular values in descending order by magnitude, and thus the columns of $\mathbf{U}$ are hierarchically ordered by how much correlation they capture in the columns of $\mathbf{X}$; similarly $\mathbf{V}$ captures correlation in the rows of $\mathbf{X}$.

Method of Snapshots

It is often impractical to construct the matrix $\mathbf{X}\mathbf{X}^*$ because of the large size of the state dimension n , let alone solve the eigenvalue problem: if $\mathbf{x}$ has a million elements, then $\mathbf{X}\mathbf{X}^*$ has a trillion elements. In 1987, Sirovich observed that it is possible to bypass this large matrix and compute the first m columns of $\mathbf{U}$ using what is now known as the method of snapshots [661].

Instead of computing the eigendecomposition of $\mathbf{X}\mathbf{X}^*$ to obtain the left singular vectors $\mathbf{U}$, we only compute the eigendecomposition of $\mathbf{X}^*\mathbf{X}$, which is much smaller and more manageable. From (1.13b), we then obtain $\mathbf{V}$ and $\hat{\mathbf{\Sigma}}$. If there are zero singular values in $\hat{\mathbf{\Sigma}}$, then we only keep the r non-zero part, $\tilde{\mathbf{\Sigma}}$, and the corresponding columns $\tilde{\mathbf{V}}$ of $\mathbf{V}$. From these matrices, it is then possible to approximate $\tilde{\mathbf{U}}$, the first r columns of $\mathbf{U}$, as follows:

$$\tilde{\mathbf{U}} = \mathbf{X}\tilde{\mathbf{V}}\tilde{\mathbf{\Sigma}}^{-1}. \quad (1.14)$$

Generalization of the Eigendecomposition

In a sense, the singular value decomposition is a generalization of the eigendecomposition that is valid for all matrices, including non-square matrices and defective square matrices that do not have a complete basis of eigenvectors.

The eigendecomposition of a diagonalizable square matrix $\mathbf{X}$ is given by

$$\mathbf{X}\mathbf{V} = \mathbf{V}\mathbf{\Lambda} \implies \mathbf{X} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}, \quad (1.15)$$

where the columns of $\mathbf{V}$ are eigenvectors and the corresponding entries of the diagonal matrix $\mathbf{\Lambda}$ are the eigenvalues. For Hermitian matrices (i.e., self-adjoint matrices such that $\mathbf{X} = \mathbf{X}^*$), the eigendecomposition takes the form

$$\mathbf{X}\mathbf{V} = \mathbf{V}\mathbf{\Lambda} \implies \mathbf{X} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^* \quad (1.16)$$

and the eigenvalues are real.

The singular value decomposition may be written in a similar form for a generic matrix $\mathbf{X}$ as

$$\mathbf{X}\mathbf{V} = \mathbf{U}\mathbf{\Sigma} \implies \mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*. \quad (1.17)$$

In this way, the singular value spectrum, given by the collection of singular values in $\mathbf{\Sigma}$, generalizes the notion of an eigenvalue spectrum, given by the collection of eigenvalues in $\mathbf{\Lambda}$. Similarly, the left and right singular vectors have an interpretation as a change of coordinates in the input space $\mathbb{C}^m$ and output space $\mathbb{C}^n$, much as the eigenvectors provide a change of coordinates to diagonalize a square matrix.

Geometric Interpretation

The columns of the matrix $\mathbf{U}$ provide an orthonormal basis for the column space of $\mathbf{X}$. Similarly, the columns of $\mathbf{V}$ provide an orthonormal basis for the row space of $\mathbf{X}$. If the columns of $\mathbf{X}$ are spatial measurements in time, then $\mathbf{U}$ encodes spatial patterns, and $\mathbf{V}$ encodes temporal patterns.

One property that makes the SVD particularly useful is the fact that both $\mathbf{U}$ and $\mathbf{V}$ are *unitary* matrices, so that $\mathbf{U}\mathbf{U}^* = \mathbf{U}^*\mathbf{U} = \mathbf{I}_{n \times n}$ and $\mathbf{V}\mathbf{V}^* = \mathbf{V}^*\mathbf{V} = \mathbf{I}_{m \times m}$. This means that solving a system of equations involving $\mathbf{U}$ or $\mathbf{V}$ is as simple as multiplication by the

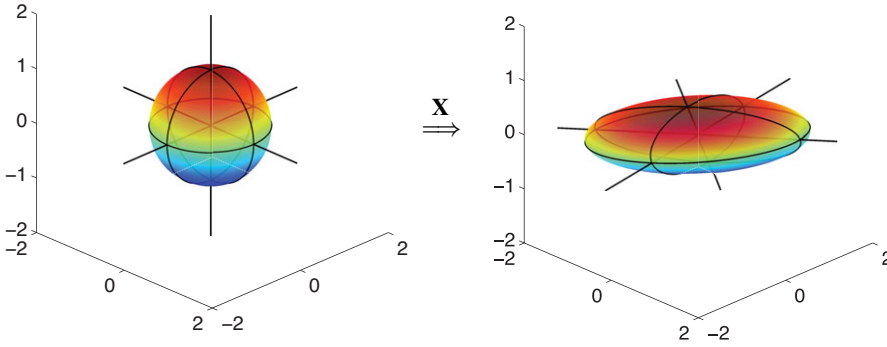


Figure 1.8 Geometric illustration of the SVD as a mapping from a sphere in $\mathbb{R}^n$ to an ellipsoid in $\mathbb{R}^m$.

transpose, which scales as $\mathcal{O}(n^2)$, as opposed to traditional methods for the generic inverse, which scale as $\mathcal{O}(n^3)$. As noted in the previous section and in [79], the SVD is intimately connected to the spectral properties of the compact self-adjoint operators $\mathbf{X}\mathbf{X}^*$ and $\mathbf{X}^*\mathbf{X}$.

The SVD of $\mathbf{X}$ may be interpreted geometrically based on how a hypersphere, given by $S^{n-1} \triangleq \{\mathbf{x} \mid \|\mathbf{x}\|_2 = 1\} \subset \mathbb{R}^n$, maps into an ellipsoid, $\{\mathbf{y} \mid \mathbf{y} = \mathbf{X}\mathbf{x} \text{ for } \mathbf{x} \in S^{n-1}\} \subset \mathbb{R}^m$, through $\mathbf{X}$. This is shown graphically in Fig. 1.8 for a sphere in $\mathbb{R}^3$ and a mapping $\mathbf{X}$ with three non-zero singular values. Because the mapping through $\mathbf{X}$ (i.e., matrix multiplication) is linear, knowing how it maps the unit sphere determines how all other vectors will map.

For the specific case shown in Fig. 1.8, we construct the matrix $\mathbf{X}$ out of three rotation matrices, $\mathbf{R}_x$, $\mathbf{R}_y$, and $\mathbf{R}_z$, and a fourth matrix to stretch out and scale the principal axes:

$$\mathbf{X} = \underbrace{\begin{bmatrix} \cos(\theta_3) & -\sin(\theta_3) & 0 \\ \sin(\theta_3) & \cos(\theta_3) & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{\mathbf{R}_z} \underbrace{\begin{bmatrix} \cos(\theta_2) & 0 & \sin(\theta_2) \\ 0 & 1 & 0 \\ -\sin(\theta_2) & 0 & \cos(\theta_2) \end{bmatrix}}_{\mathbf{R}_y} \times \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta_1) & -\sin(\theta_1) \\ 0 & \sin(\theta_1) & \cos(\theta_1) \end{bmatrix}}_{\mathbf{R}_x} \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & \sigma_3 \end{bmatrix}.$$

In this case, $\theta_1 = \pi/15$, $\theta_2 = -\pi/9$, and $\theta_3 = -\pi/20$, and $\sigma_1 = 3$, $\sigma_2 = 1$, and $\sigma_3 = 0.5$. These rotation matrices do not commute, and so the order of rotation matters. If one of the singular values is zero, then a dimension is removed and the ellipsoid collapses onto a lower-dimensional subspace. The product $\mathbf{R}_x\mathbf{R}_y\mathbf{R}_z$ is the unitary matrix $\mathbf{U}$ in the SVD of $\mathbf{X}$. The matrix $\mathbf{V}$ is the identity. Codes to reproduce this example in MATLAB and in Python are provided on the book's GitHub.

Invariance of the SVD to Unitary Transformations

A useful property of the SVD is that if we left- or right-multiply our data matrix $\mathbf{X}$ by a unitary transformation, it preserves the terms in the SVD, except for the corresponding left or right unitary matrix $\mathbf{U}$ or $\mathbf{V}$, respectively. This has important implications, since the discrete Fourier transform (DFT; see Chapter 2) $\mathcal{F}$ is a unitary transform, meaning that the

SVD of data $\hat{\mathbf{X}} = \mathcal{F}\mathbf{X}$ will be exactly the same as the SVD of $\mathbf{X}$, except that the modes $\hat{\mathbf{U}}$ will be the DFT of modes $\mathbf{U}$: $\hat{\mathbf{U}} = \mathcal{F}\mathbf{U}$. In addition, the invariance of the SVD to unitary transformations enables the use of compressed measurements to reconstruct SVD modes that are sparse in some transform basis (see [Chapter 3](#)).

The invariance of SVD to unitary transformations is geometrically intuitive, as unitary transformations rotate vectors in space, but do not change their inner products or correlation structures. We denote a left unitary transformation by $\mathbf{C}$, so that $\mathbf{Y} = \mathbf{C}\mathbf{X}$, and a right unitary transformation by $\mathbf{P}^*$, so that $\mathbf{Y} = \mathbf{X}\mathbf{P}^*$. The SVD of $\mathbf{X}$ will be denoted $\mathbf{U}_\mathbf{X}\boldsymbol{\Sigma}_\mathbf{X}\mathbf{V}_\mathbf{X}^*$ and the SVD of $\mathbf{Y}$ will be $\mathbf{U}_\mathbf{Y}\boldsymbol{\Sigma}_\mathbf{Y}\mathbf{V}_\mathbf{Y}^*$.

Left Unitary Transformations

First, consider a left unitary transformation of $\mathbf{X}$: $\mathbf{Y} = \mathbf{C}\mathbf{X}$. Computing the correlation matrix $\mathbf{Y}^*\mathbf{Y}$, we find

$$\mathbf{Y}^*\mathbf{Y} = \mathbf{X}^*\mathbf{C}^*\mathbf{C}\mathbf{X} = \mathbf{X}^*\mathbf{X}. \quad (1.18)$$

The projected data has the same eigendecomposition, resulting in the same $\mathbf{V}_\mathbf{X}$ and $\boldsymbol{\Sigma}_\mathbf{X}$. Using the method of snapshots to reconstruct $\mathbf{U}_\mathbf{Y}$, we find

$$\mathbf{U}_\mathbf{Y} = \mathbf{Y}\mathbf{V}_\mathbf{X}\boldsymbol{\Sigma}_\mathbf{X}^{-1} = \mathbf{C}\mathbf{X}\mathbf{V}_\mathbf{X}\boldsymbol{\Sigma}_\mathbf{X}^{-1} = \mathbf{C}\mathbf{U}_\mathbf{X}. \quad (1.19)$$

Thus, $\mathbf{U}_\mathbf{Y} = \mathbf{C}\mathbf{U}_\mathbf{X}$, $\boldsymbol{\Sigma}_\mathbf{Y} = \boldsymbol{\Sigma}_\mathbf{X}$, and $\mathbf{V}_\mathbf{Y} = \mathbf{V}_\mathbf{X}$. The SVD of $\mathbf{Y}$ is then

$$\mathbf{Y} = \mathbf{C}\mathbf{X} = \mathbf{C}\mathbf{U}_\mathbf{X}\boldsymbol{\Sigma}_\mathbf{X}\mathbf{V}_\mathbf{X}^*. \quad (1.20)$$

Right Unitary Transformations

For a right unitary transformation $\mathbf{Y} = \mathbf{X}\mathbf{P}^*$, the correlation matrix $\mathbf{Y}^*\mathbf{Y}$ is

$$\mathbf{Y}^*\mathbf{Y} = \mathbf{P}\mathbf{X}^*\mathbf{X}\mathbf{P}^* = \mathbf{P}\mathbf{V}_\mathbf{X}\boldsymbol{\Sigma}_\mathbf{X}^2\mathbf{V}_\mathbf{X}^*\mathbf{P}^*, \quad (1.21)$$

with the following eigendecomposition:

$$\mathbf{Y}^*\mathbf{Y}\mathbf{P}\mathbf{V}_\mathbf{X} = \mathbf{P}\mathbf{V}_\mathbf{X}\boldsymbol{\Sigma}_\mathbf{X}^2. \quad (1.22)$$

Thus, $\mathbf{V}_\mathbf{Y} = \mathbf{P}\mathbf{V}_\mathbf{X}$ and $\boldsymbol{\Sigma}_\mathbf{Y} = \boldsymbol{\Sigma}_\mathbf{X}$. We may use the method of snapshots to reconstruct $\mathbf{U}_\mathbf{Y}$:

$$\mathbf{U}_\mathbf{Y} = \mathbf{Y}\mathbf{P}\mathbf{V}_\mathbf{X}\boldsymbol{\Sigma}_\mathbf{X}^{-1} = \mathbf{X}\mathbf{V}_\mathbf{X}\boldsymbol{\Sigma}_\mathbf{X}^{-1} = \mathbf{U}_\mathbf{X}. \quad (1.23)$$

Thus, $\mathbf{U}_\mathbf{Y} = \mathbf{U}_\mathbf{X}$, and we may write the SVD of $\mathbf{Y}$ as

$$\mathbf{Y} = \mathbf{X}\mathbf{P}^* = \mathbf{U}_\mathbf{X}\boldsymbol{\Sigma}_\mathbf{X}\mathbf{V}_\mathbf{X}^*\mathbf{P}^*. \quad (1.24)$$

1.4 Pseudo-Inverse, Least-Squares, and Regression

Many physical systems may be represented as a linear system of equations,

$$\mathbf{A}\mathbf{x} = \mathbf{b}, \quad (1.25)$$

where the constraint matrix $\mathbf{A}$ and vector $\mathbf{b}$ are known, and the vector $\mathbf{x}$ is unknown. If $\mathbf{A}$ is a square, invertible matrix (i.e., $\mathbf{A}$ has non-zero determinant), then there exists a unique solution $\mathbf{x}$ for every $\mathbf{b}$. However, when $\mathbf{A}$ is either singular or rectangular, there may be one, none, or infinitely many solutions, depending on the specific $\mathbf{b}$ and the column and row spaces of $\mathbf{A}$.

First, consider the *under-determined system*, where $\mathbf{A} \in \mathbb{C}^{n \times m}$ and $n \ll m$ (i.e., $\mathbf{A}$ is a short-fat matrix), so that there are fewer equations than unknowns. This type of system is likely to have columns that span all of $\mathbb{R}^n$, since it has many more columns than are required for a linearly independent basis.⁴ Generically, if a short-fat $\mathbf{A}$ has n linearly independent columns (i.e., its column space spans $\mathbb{R}^n$), then there are infinitely many solutions $\mathbf{x}$ for every $\mathbf{b}$. The system is called *under-determined* because there are not enough values in $\mathbf{b}$ to uniquely determine the higher-dimensional $\mathbf{x}$.

Similarly, consider the *over-determined system*, where $n \gg m$ (i.e., a tall-skinny matrix), so that there are more equations than unknowns. This matrix cannot have n linearly independent columns, and so it is guaranteed that there are vectors $\mathbf{b}$ that have no solution $\mathbf{x}$. In fact, there will only be a solution $\mathbf{x}$ if $\mathbf{b}$ is in the column space of $\mathbf{A}$, i.e., $\mathbf{b} \in \text{col}(\mathbf{A})$.

Technically, there may be some choices of $\mathbf{b}$ that admit infinitely many solutions $\mathbf{x}$ for a tall-skinny matrix $\mathbf{A}$ and other choices of $\mathbf{b}$ that admit zero solutions even for a short-fat matrix. The solution space to the system in (1.25) is determined by the following four fundamental subspaces of $\mathbf{A} = \tilde{\mathbf{U}}\tilde{\Sigma}\tilde{\mathbf{V}}^*$, where the rank r is chosen to include all non-zero singular values:

- The column space, $\text{col}(\mathbf{A})$, is the span of the columns of $\mathbf{A}$, also known as the *range*. The column space of $\mathbf{A}$ is the same as the column space of $\tilde{\mathbf{U}}$.
- The orthogonal complement to $\text{col}(\mathbf{A})$ is $\ker(\mathbf{A}^*)$, given by the column space of $\hat{\mathbf{U}}^\perp$ from Fig. 1.1.
- The row space, $\text{row}(\mathbf{A})$, is the span of the rows of $\mathbf{A}$, which is spanned by the columns of $\tilde{\mathbf{V}}$. The row space of $\mathbf{A}$ is equal to $\text{row}(\mathbf{A}) = \text{col}(\mathbf{A}^*)$.
- The kernel space, $\ker(\mathbf{A})$, is the orthogonal complement to $\text{row}(\mathbf{A})$, and is also known as the *null space*. The null space is the subspace of vectors that map through $\mathbf{A}$ to zero, i.e., $\mathbf{A}\mathbf{x} = \mathbf{0}$, given by $\text{col}(\hat{\mathbf{V}}^\perp)$.

More precisely, if $\mathbf{b} \in \text{col}(\mathbf{A})$ and if $\dim(\ker(\mathbf{A})) \neq 0$, then there are infinitely many solutions $\mathbf{x}$. Note that the condition $\dim(\ker(\mathbf{A})) \neq 0$ is guaranteed for a short-fat matrix. Similarly, if $\mathbf{b} \notin \text{col}(\mathbf{A})$, then there are no solutions, and the system of equations in (1.25) is called *inconsistent*.

The fundamental subspaces above satisfy the following properties:

$$\text{col}(\mathbf{A}) \oplus \ker(\mathbf{A}^*) = \mathbb{R}^n, \quad (1.26a)$$

$$\text{col}(\mathbf{A}^*) \oplus \ker(\mathbf{A}) = \mathbb{R}^n. \quad (1.26b)$$

Remark 1.1 *There is an extensive literature on random matrix theory, where the above stereotypes are almost certainly true, meaning that they are true with high probability. For example, a system $\mathbf{A}\mathbf{x} = \mathbf{b}$ is extremely unlikely to have a solution for a random matrix $\mathbf{A} \in \mathbb{R}^{n \times m}$ and random vector $\mathbf{b} \in \mathbb{R}^n$ with $n \gg m$, since there is little chance that $\mathbf{b}$ is in the column space of $\mathbf{A}$. These properties of random matrices will play a prominent role in compressed sensing (see Chapter 3).*

⁴ It is easy to construct degenerate examples where the columns of a short-fat matrix do not form a full basis for $\mathbb{R}^n$, such as

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}.$$

In the over-determined case when no solution exists, we would often like to find the solution $\mathbf{x}$ that minimizes the sum-squared error $\|\mathbf{Ax} - \mathbf{b}\|_2^2$, the so-called *least-squares* solution. Note that the least-squares solution also minimizes $\|\mathbf{Ax} - \mathbf{b}\|_2$. In the under-determined case when infinitely many solutions exist, we may like to find the solution $\mathbf{x}$ with minimum norm $\|\mathbf{x}\|_2$ so that $\mathbf{Ax} = \mathbf{b}$, the so-called *minimum-norm* solution.

The SVD is the technique of choice for these important optimization problems. First, if we substitute an exact truncated SVD $\mathbf{A} = \tilde{\mathbf{U}}\tilde{\mathbf{\Sigma}}\tilde{\mathbf{V}}^*$ in for $\mathbf{A}$, we can “invert” each of the matrices $\tilde{\mathbf{U}}$, $\tilde{\mathbf{\Sigma}}$, and $\tilde{\mathbf{V}}^*$ in turn, resulting in the Moore–Penrose *left pseudo-inverse* [558, 559, 602, 773] $\mathbf{A}^\dagger$ of $\mathbf{A}$:

$$\mathbf{A}^\dagger \triangleq \tilde{\mathbf{V}}\tilde{\mathbf{\Sigma}}^{-1}\tilde{\mathbf{U}}^* \implies \mathbf{A}^\dagger\mathbf{A} = \tilde{\mathbf{V}}\tilde{\mathbf{V}}^*. \quad (1.27)$$

Note that $\mathbf{A}^\dagger\mathbf{A}$ will only equal the identity $\mathbf{I}_{m \times m}$ if the truncated SVD captures all non-zero singular values; otherwise $\tilde{\mathbf{V}}\tilde{\mathbf{V}}^* \neq \mathbf{I}_{m \times m}$, and it will only approximate the identity. This may be used to find both the minimum-norm and least-squares solutions to (1.25):

$$\mathbf{A}^\dagger\mathbf{Ax} = \mathbf{A}^\dagger\mathbf{b} \implies \tilde{\mathbf{x}} = \tilde{\mathbf{V}}\tilde{\mathbf{\Sigma}}^{-1}\tilde{\mathbf{U}}^*\mathbf{b}. \quad (1.28)$$

Plugging the solution $\tilde{\mathbf{x}}$ back in to (1.25) results in

$$\mathbf{Ax} = \tilde{\mathbf{U}}\tilde{\mathbf{\Sigma}}\tilde{\mathbf{V}}^*\tilde{\mathbf{V}}\tilde{\mathbf{\Sigma}}^{-1}\tilde{\mathbf{U}}^*\mathbf{b} \quad (1.29a)$$

$$= \tilde{\mathbf{U}}\tilde{\mathbf{U}}^*\mathbf{b}. \quad (1.29b)$$

Although $\mathbf{U}^*\mathbf{U} = \mathbf{UU}^* = \mathbf{I}_{n \times n}$ for the exact SVD, $\tilde{\mathbf{U}}\tilde{\mathbf{U}}^*$ is not necessarily the identity matrix for a truncated basis of left singular vectors $\tilde{\mathbf{U}}$, but is rather a projection onto the column space of $\tilde{\mathbf{U}}$. Therefore, $\tilde{\mathbf{x}}$ will only be an exact solution to (1.25) when $\mathbf{b}$ is in the column space of $\tilde{\mathbf{U}}$, and therefore in the column space of $\mathbf{A}$. Assuming that $\tilde{\mathbf{U}}\tilde{\mathbf{U}}^*$ is equal to the identity is one of the most common accidental misuses of the SVD.⁵ However, it is still true that $\tilde{\mathbf{U}}^*\tilde{\mathbf{U}} = \mathbf{I}_{r \times r}$, where r is the rank of $\mathbf{A}$.

Computing the pseudo-inverse $\mathbf{A}^\dagger$ is computationally efficient, after the expensive up-front cost of computing the SVD. Inverting the unitary matrices $\tilde{\mathbf{U}}$ and $\tilde{\mathbf{V}}$ involves matrix multiplications by the transpose matrices, which are $\mathcal{O}(n^2)$ operations. Inverting $\tilde{\mathbf{\Sigma}}$ is even more efficient, since it is a diagonal matrix, requiring $\mathcal{O}(n)$ operations. In contrast, inverting a dense square matrix would require an $\mathcal{O}(n^3)$ operation.

Condition Number

The condition number of a matrix $\mathbf{A}$ is a measure of how sensitive matrix multiplication and inversion are to errors in the input. Larger condition number indicates higher sensitivity and worse performance. The condition number $\kappa(\mathbf{A})$ is directly related to the singular values of the matrix:

$$\kappa(\mathbf{A}) = \frac{\sigma_{\max}(\mathbf{A})}{\sigma_{\min}(\mathbf{A})}. \quad (1.30)$$

The condition number is a central concept in all of numerical linear algebra and applied computation. It is easiest to understand the effect of a large condition number when

⁵ The authors are not immune to this, having mistakenly used this fictional identity in an early version of [134].

considering the linear system of equations $\mathbf{Ax} = \mathbf{b}$. If the vector $\mathbf{x}$ is not specified perfectly, but instead has some error $\epsilon_{\mathbf{x}}$, then the system becomes

$$\mathbf{A}(\mathbf{x} + \epsilon_{\mathbf{x}}) = \mathbf{b} + \epsilon_{\mathbf{b}}, \quad (1.31)$$

where $\epsilon_{\mathbf{b}}$ is the corresponding error in $\mathbf{b}$. If we assume a worst-case scenario, where $\epsilon_{\mathbf{x}}$ is aligned with the singular vector corresponding to the maximum singular value $\sigma_{\max}$, and the vector $\mathbf{x}$ is aligned with the singular vector corresponding to the minimum singular value $\sigma_{\min}$, then the output is

$$\mathbf{A}(\mathbf{x} + \epsilon_{\mathbf{x}}) = \underbrace{\sigma_{\min} \mathbf{x}}_{\mathbf{b}} + \underbrace{\sigma_{\max} \epsilon_{\mathbf{x}}}_{\epsilon_{\mathbf{b}}}. \quad (1.32)$$

The output signal-to-noise $\|\mathbf{b}\|/\|\epsilon_{\mathbf{b}}\|$ is equal to the input signal-to-noise $\|\mathbf{x}\|/\|\epsilon_{\mathbf{x}}\|$ multiplied by a factor of $\sigma_{\min}/\sigma_{\max}$. In other words, the signal-to-noise of the output has been reduced by a factor equal to the condition number $\kappa(\mathbf{A})$. Even if the vectors $\mathbf{x}$ and $\epsilon_{\mathbf{x}}$ are not perfectly aligned with the worst-case directions, they are likely to have some component of all of the singular vector directions. In this case, components of the error will still experience large amplification relative to other components of the desired output.

A similar issue arises when solving for $\mathbf{x}$ given an imperfectly specified $\mathbf{b}$ with some error $\epsilon_{\mathbf{b}}$. Now the worst-case scenario is where $\epsilon_{\mathbf{b}}$ is aligned with the singular vector corresponding to the minimum singular value $\sigma_{\min}$, and the vector $\mathbf{b}$ is aligned with the singular vector corresponding to the maximum singular value $\sigma_{\max}$. Then the estimated solution for $\mathbf{x} + \epsilon_{\mathbf{x}}$ is

$$\mathbf{x} + \epsilon_{\mathbf{x}} \approx \mathbf{A}^{\dagger}(\mathbf{b} + \epsilon_{\mathbf{b}}) = \frac{1}{\sigma_{\max}} \mathbf{b} + \frac{1}{\sigma_{\min}} \epsilon_{\mathbf{b}}. \quad (1.33)$$

The signal-to-noise of the estimated $\mathbf{x}$ has also been reduced, or degraded, by a factor equal to the condition number $\kappa(\mathbf{A})$.

One approach to mitigate a large condition number is to truncate the SVD more aggressively, essentially increasing the effective minimum singular value $\sigma_{\min}$. However, this comes at the cost of decreasing the size of the subspace $\tilde{\mathbf{U}}$ used to approximate the output.

```
>> kappanew = 1.e-5; % Desired condition number
>> [U,S,V] = svd(A,'econ')
>> r = max(find(diag(S)>max(S(:))*kappanew));
>> invA = V(:,1:r)*inv(S(1:r,1:r))*U(:,1:r)'; % Approximate
```

One-Dimensional Linear Regression

Regression is an important statistical tool to relate variables to one another based on data [476]. Consider the collection of data in Fig. 1.9. The red crosses are obtained by adding Gaussian white noise to the black line, as shown in Code 1.2. We assume that the data is linearly related, as in (1.25), and we use the pseudo-inverse to find the least-squares solution for the slope x below (blue dashed line), shown in Code 1.2:

$$\begin{bmatrix} \mathbf{b} \\ \mathbf{1} \end{bmatrix} = \begin{bmatrix} \mathbf{a} \\ \mathbf{1} \end{bmatrix} x = \tilde{\mathbf{U}} \tilde{\Sigma} \tilde{\mathbf{V}}^* x. \quad (1.34a)$$

$$\Rightarrow x = \tilde{\mathbf{V}} \tilde{\Sigma}^{-1} \tilde{\mathbf{U}}^* \mathbf{b}. \quad (1.34b)$$

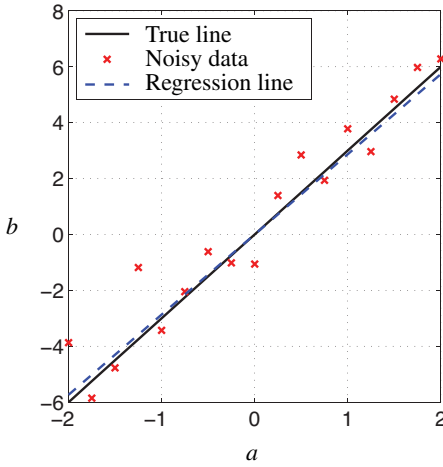


Figure 1.9 Illustration of linear regression using noisy data.

In (1.34b), $\tilde{\Sigma} = \|\mathbf{a}\|_2$, $\tilde{\mathbf{V}} = 1$, and $\tilde{\mathbf{U}} = \mathbf{a}/\|\mathbf{a}\|_2$. Taking the left pseudo-inverse:

$$x = \frac{\mathbf{a}^* \mathbf{b}}{\|\mathbf{a}\|_2^2}. \quad (1.35)$$

This makes physical sense, if we think of x as the value that best maps our vector $\mathbf{a}$ to the vector $\mathbf{b}$. Then, the best single value x is obtained by taking the dot product of $\mathbf{b}$ with the normalized $\mathbf{a}$ direction. We then add a second normalization factor $\|\mathbf{a}\|_2$ because the $\mathbf{a}$ in (1.34a) is not normalized.

Note that strange things happen if you use row vectors instead of column vectors in (1.34) above. Also, if the noise magnitude becomes large relative to the slope x , the pseudo-inverse will undergo a phase change in accuracy, related to the hard-thresholding results in subsequent sections.

Code 1.2 [MATLAB] Least-squares fit of noisy data in Fig. 1.9.

```
% Generate noisy data
x = 3; % True slope
a = [-2:.25:2]';
b = a*x + 1*randn(size(a)); % Add noise
plot(a,x*a,'k') % True relationship
hold on, plot(a,b,'rx') % Noisy measurements

% Compute least-squares approximation with the SVD
[U,S,V] = svd(a,'econ');
xtilde = V*inv(S)*U'*b; % Least-square fit
plot(a,xtilde*a,'b--') % Plot fit

%% Alternative formulations of least-squares
xtilde1 = V*inv(S)*U'*b
xtilde2 = pinv(a)*b
xtilde3 = regress(b,a)
```

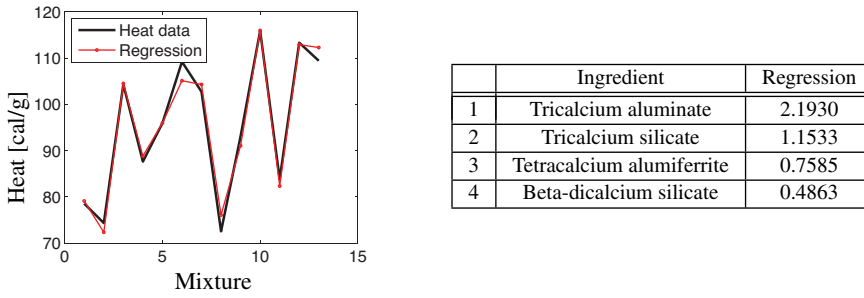


Figure 1.10 Heat data for cement mixtures containing four basic ingredients.

Code 1.2 [Python] Least-squares fit of noisy data in Fig. 1.9.

```
x = 3 # True slope
a = np.arange(-2,2,0.25)
a = a.reshape(-1, 1)
b = x*a + np.random.randn(*a.shape) # Add noise
plt.plot(a, x*a, Color='k', LineWidth=2, label='True line')
# True relationship
plt.plot(a, b, 'x', Color='r', MarkerSize = 10, label='Noisy
data') # Noisy measurements

# Compute least-squares approximation with the SVD
U, S, VT = np.linalg.svd(a,full_matrices=False)
xtilde = VT.T @ np.linalg.inv(np.diag(S)) @ U.T @ b # Least-
square fit
plt.plot(a,xtilde * a, '--',Color='b',LineWidth=4, label='
Regression line')

# Alternative formulations of least squares
xtilde1 = VT.T @ np.linalg.inv(np.diag(S)) @ U.T @ b
xtilde2 = np.linalg.pinv(a) @ b
```

Multi-Linear Regression

Example 1: Cement Heat Generation Data

First, we begin with a simple built-in MATLAB data set that describes the heat generation for various cement mixtures that comprise four basic ingredients (see Fig. 1.10). In this problem, we are solving (1.25), where $\mathbf{A} \in \mathbb{R}^{13 \times 4}$, since there are four ingredients and heat measurements for 13 unique mixtures. The goal is to determine the weighting $\mathbf{x}$ that relates the proportions of the four ingredients to the heat generation. It is possible to find the minimum error solution using the SVD, as shown in Code 1.3. Alternatives, using **regress** and **pinv**, are also explored.

Code 1.3 [MATLAB] Multi-linear regression for cement heat data.

```
load hald; % Load Portland Cement dataset
A = ingredients;
b = heat;

[U,S,V] = svd(A,'econ');
x = V*inv(S)*U'*b; % Solve Ax=b using the SVD
```

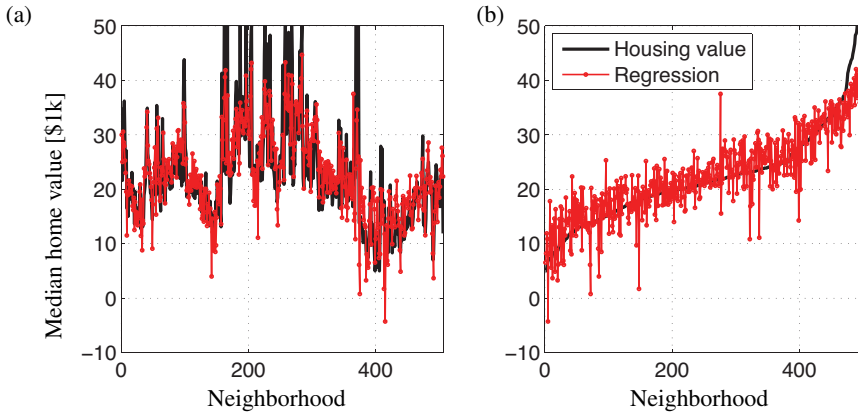


Figure 1.11 Multi-linear regression of home prices using various factors: (a) unsorted data and (b) data sorted by home value.

```

plot(b, 'k'); hold on      % Plot data
plot(A*x, 'r-o');         % Plot fit

x = regress(b,A);         % Alternative 1 (regress)
x = pinv(A)*b;            % Alternative 2 (pinv)

```

Code 1.3 [Python] Multi-linear regression for cement heat data.

```

# Load dataset
A = np.loadtxt(os.path.join('.', 'DATA', 'hald_ingredients.
    csv'), delimiter=',')
b = np.loadtxt(os.path.join('.', 'DATA', 'hald_heat.csv'),
    delimiter=',')

# Solve Ax=b using SVD
U, S, VT = np.linalg.svd(A, full_matrices=0)
x = VT.T @ np.linalg.inv(np.diag(S)) @ U.T @ b

plt.plot(b, Color='k', LineWidth=2, label='Heat Data')
plt.plot(A*x, '-o', Color='r', label='Regression')

x = np.linalg.pinv(A)*b # Alternative

```

Example 2: Boston Housing Data

In this example, we explore a larger data set to determine which factors best predict prices in the Boston housing market [312]. This data is available from the UCI Machine Learning Repository [33].

There are 13 attributes that are correlated with house price, such as per-capita crime rate and property-tax rate. These features are regressed onto the price data, the best-fit price prediction is plotted against the true house value in Fig. 1.11, and the regression coefficients are shown in Fig. 1.12. Although the house value is not perfectly predicted, the trend agrees quite well. It is often the case that the highest value outliers are not well captured by simple linear fits, as in this example.

This data contains prices and attributes for 506 homes, so the attribute matrix is of size 506×13 . It is important to pad this matrix with an additional column of ones, to take

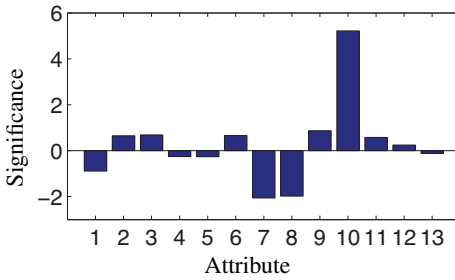


Figure 1.12 Significance of various attributes in the regression.

into account the possibility of a non-zero constant offset in the regression formula. This corresponds to the “y intercept” in a simple one-dimensional linear regression. The code for this example is nearly identical to the example above, and is available on the book’s GitHub.

1.5 Principal Component Analysis (PCA)

Principal component analysis (PCA) is one of the central applications of the SVD, providing a statistical interpretation of the data-driven, hierarchical coordinate system used to represent high-dimensional correlated data. This coordinate system involves the correlation matrices described in [Section 1.3](#). Importantly, PCA pre-processes the data by mean subtraction and setting the variance to unity before performing the SVD. The geometry of the resulting coordinate system is determined by principal components (PCs) that are uncorrelated (orthogonal) to each other, but have maximal correlation with the measurements. This theory was developed in 1901 by Pearson [550], and independently by Hotelling in the 1930s [338, 339]. Jolliffe [351] provides a good reference text.

Often in statistics, a number of measurements are collected in a single experiment, and these measurements are typically arranged into a row vector. The measurements may be features of an observable, such as demographic features of a specific human individual. A number of experiments are conducted, and each measurement vector is arranged as a row in a large matrix $\mathbf{X}$, resembling the structure of how data is recorded in a spreadsheet. In the example of demography, the collection of experiments may be gathered via polling. Note that this convention for $\mathbf{X}$, consisting of rows of features, is different than the convention throughout the remainder of this chapter, where individual feature “snapshots” are arranged as columns. However, we choose to be consistent with PCA literature in this section. The matrix will still be size $n \times m$, although it may have more rows than columns, or vice versa.

Computation

We now compute the average row $\bar{\mathbf{x}}$ (i.e., the mean of all rows), and subtract it from $\mathbf{X}$. The mean $\bar{\mathbf{x}}$ is given by

$$\bar{x}_j = \frac{1}{n} \sum_{i=1}^n \mathbf{X}_{ij}, \quad (1.36)$$

and the mean matrix is

$$\bar{\mathbf{X}} = \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix} \bar{\mathbf{x}}. \quad (1.37)$$

Subtracting $\bar{\mathbf{X}}$ from $\mathbf{X}$ results in the mean-subtracted data $\mathbf{B}$:

$$\mathbf{B} = \mathbf{X} - \bar{\mathbf{X}}. \quad (1.38)$$

The covariance matrix of $\mathbf{B}$ is given by

$$\mathbf{C} = \frac{1}{n-1} \mathbf{B}^* \mathbf{B}. \quad (1.39)$$

Note that the covariance is normalized by $n-1$ instead of n , even though there are n sample points. This is known as Bessel's correction, which compensates for the fact that the sample variance is biased because it does not capture the variance of the sample mean $\bar{\mathbf{X}}$ about the true mean. The covariance matrix $\mathbf{C}$ is symmetric and positive semi-definite, having non-negative real eigenvalues. Each entry C_{ij} quantifies the correlation of the i and j features across all experiments.

The principal components are the eigenvectors of $\mathbf{C}$, and they define a change of coordinates in which the covariance matrix is diagonal:

$$\mathbf{C}\mathbf{V} = \mathbf{V}\mathbf{D} \implies \mathbf{C} = \mathbf{V}\mathbf{D}\mathbf{V}^* \implies \mathbf{D} = \mathbf{V}^*\mathbf{C}\mathbf{V}. \quad (1.40)$$

The columns of the eigenvector matrix $\mathbf{V}$ are the principal components, and the elements of the diagonal matrix $\mathbf{D}$ are the variances of the data along these directions. This transformation is guaranteed to exist, since $\mathbf{C}$ is Hermitian and the columns of $\mathbf{V}$ are orthonormal. In these principal component coordinates, all features are linearly uncorrelated with each other.

The matrix of principal components $\mathbf{V}$ is also the matrix of right singular vectors of $\mathbf{B}$. Substituting $\mathbf{B} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$ into (1.39) and comparing with (1.40) yields

$$\mathbf{C} = \frac{1}{n-1} \mathbf{B}^* \mathbf{B} = \frac{1}{n-1} \mathbf{V} \mathbf{\Sigma}^2 \mathbf{V}^* \implies \mathbf{D} = \frac{1}{n-1} \mathbf{\Sigma}^2. \quad (1.41)$$

The variance of the data in these coordinates, given by the diagonal elements λ_k of $\mathbf{D}$, is related to the singular values as

$$\lambda_k = \frac{\sigma_k^2}{n-1}. \quad (1.42)$$

Thus, the SVD provides a numerically robust approach for computing the principal components. An approximation $\tilde{\mathbf{B}}$ obtained by keeping only the first r principal components will have a missing variance related to the squared Frobenius norm error in (1.7).

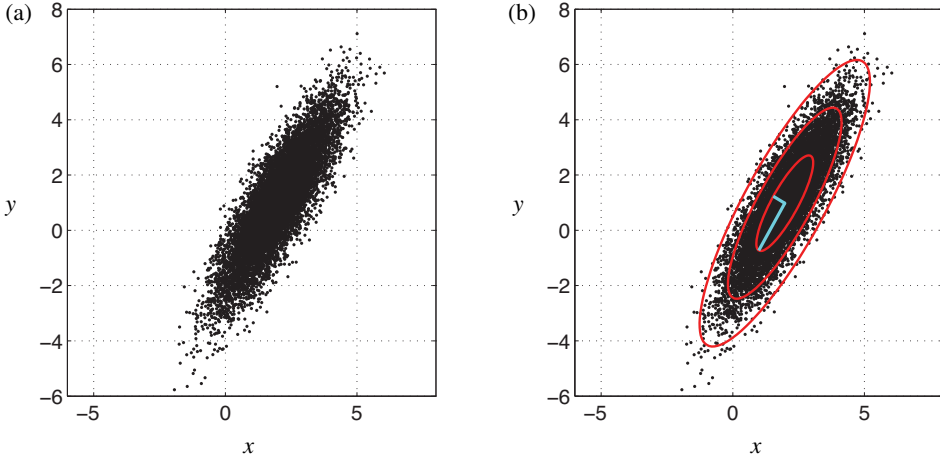
The `pca` Command

In MATLAB, there are the additional commands `pca` and `princomp` (based on `pca`) for the principal component analysis:

```
>> [V, score, s2] = pca(X);
```

Table 1.1. Standard deviation of data and normalized singular values.

	σ_1	σ_2
Data	2	0.5
SVD	1.974	0.503

**Figure 1.13** (a) Principal components capture the variance of mean-subtracted Gaussian data. (b) The first three standard deviation ellipsoids (red), and the two left singular vectors, scaled by singular values ($\sigma_1 v_1 + x_C$ and $\sigma_2 v_2 + x_C$, cyan), are shown.

The matrix $\mathbf{V}$ is equivalent to the $\mathbf{V}$ matrix from the SVD of $\mathbf{B} = \mathbf{X} - \bar{\mathbf{X}}$, up to sign changes of the columns. The vector $\mathbf{s2}$ contains eigenvalues of the covariance of $\mathbf{B}$, also known as principal component variances; these values are the squares of the singular values. The variable `score` simply contains the coordinates of each row of $\mathbf{B}$ (the mean-subtracted data) in the principal component directions. In general, we often prefer to use the `svd` command with the various pre-processing steps described above.

Example: Noisy Gaussian Data

Consider the noisy cloud of data in Fig. 1.13(a), generated using Code 1.4. The data is generated by selecting 10000 vectors from a two-dimensional normal distribution with zero mean and unit variance. These vectors are then scaled in the x and y directions by the values in Table 1.1 and rotated by $\pi/3$. Finally, the entire cloud of data is translated so that it has a non-zero center $\mathbf{x}_C = [2 \ 1]^T$.

Using Code 1.4, the PCA is performed and used to plot confidence intervals using multiple standard deviations, shown in Fig. 1.13(b). The singular values, shown in Table 1.1, match the data scaling. The matrix $\mathbf{U}$ from the SVD also closely matches the rotation matrix, up to a sign on the rows:

$$\mathbf{R}_{\pi/3} = \begin{bmatrix} 0.5 & 0.8660 \\ -0.8660 & 0.5 \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} -0.4998 & -0.8662 \\ -0.8662 & 0.4998 \end{bmatrix}.$$

Note that $\mathbf{R}_{\pi/3}$ is a rotation matrix designed to rotate row vectors by multiplication on the right by $\mathbf{R}_{\pi/3}$. For rotation of column vectors by left multiplication, this matrix would be transposed.

Code 1.4 [MATLAB] PCA example on noisy cloud of data.

```
% Generate noisy cloud of data
xC = [2, 1]; % Center of data (mean)
sig = [2, .5]; % Principal axes

theta = pi/3; % Rotate cloud by pi/3
R = [cos(theta) sin(theta); % Rotation matrix
     -sin(theta) cos(theta)];

nPoints = 10000; % Create 10,000 points
X = randn(nPoints,2)*diag(sig)*R + ones(nPoints,2)*diag(xC);
scatter(X(:,1),X(:,2),'k.','LineWidth',2) % Plot data

% Compute PCA and plot confidence intervals
Xavg = mean(X,1); % Compute mean
B = X - ones(nPoints,1)*Xavg; % Mean-subtracted Data
[U,S,V] = svd(B/sqrt(nPoints),'econ'); % PCA via SVD

theta = (0:.01:1)*2*pi;
Xstd = [cos(theta') sin(theta')] * S * V'; % 1std conf. interval
hold on, plot(Xavg(1)+Xstd(:,1),Xavg(2) + Xstd(:,2),'r-')
plot(Xavg(1)+2*Xstd(:,1),Xavg(2) + 2*Xstd(:,2),'r-')
plot(Xavg(1)+3*Xstd(:,1),Xavg(2) + 3*Xstd(:,2),'r-')
```

Code 1.4 [Python] PCA example on noisy cloud of data.

```
# Generate noisy cloud of data
xC = np.array([2, 1]) # Center of data (mean)
sig = np.array([2, 0.5]) # Principal axes

theta = np.pi/3 # Rotate cloud by pi/3
R = np.array([[np.cos(theta), -np.sin(theta)], # Rotation mat
              [np.sin(theta), np.cos(theta)]])

nPoints = 10000 # Create 10,000 points
X = R @ np.diag(sig) @ np.random.randn(2,nPoints) + np.diag(
    xC) @ np.ones((2,nPoints))
ax1.plot(X[0,:],X[1,:], '.', Color='k') # Plot data
Xavg = np.mean(X,axis=1) # Compute mean
B = X - np.tile(Xavg,(nPoints,1)).T # Mean-subtracted data

# Find principal components (SVD)
U, S, VT = np.linalg.svd(B/np.sqrt(nPoints),full_matrices=0)
theta = 2 * np.pi * np.arange(0,1,0.01)
Xstd = U @ np.diag(S) @ np.array([np.cos(theta),np.sin(theta)
    ])
ax2.plot(Xavg[0] + Xstd[0,:], Xavg[1] + Xstd[1,:], '--',color=
    'r',LineWidth=3)
ax2.plot(Xavg[0] + 2*Xstd[0,:], Xavg[1] + 2*Xstd[1,:], '--',
    color='r',LineWidth=3)
ax2.plot(Xavg[0] + 3*Xstd[0,:], Xavg[1] + 3*Xstd[1,:], '--',
    color='r',LineWidth=3)
```

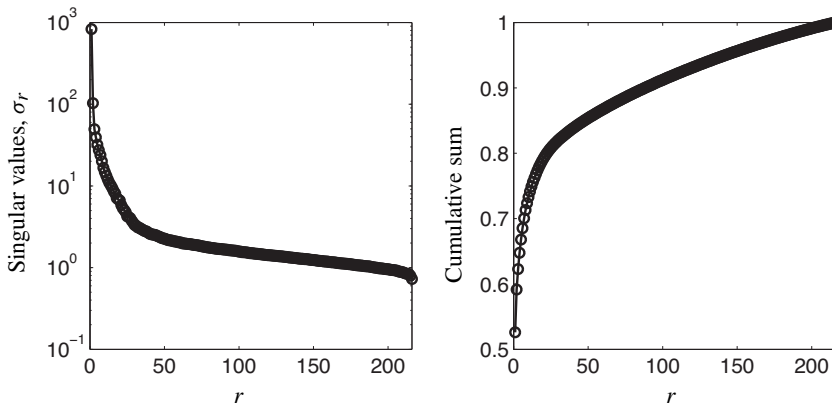


Figure 1.14 Singular values for the ovarian cancer data.

Finally, it is also possible to compute using the **pca** command in MATLAB:

```
>> [V,score,s2] = pca(X);
>> norm(score*V - B)

ans =
    1.4900e-13
```

Example: Ovarian Cancer Data

The ovarian cancer data set, which is built into MATLAB, provides a more realistic example to illustrate the benefits of PCA. This example consists of gene data for 216 patients, 121 of whom have ovarian cancer, and 95 of whom do not. For each patient, there is a vector of data containing the expression of 4000 genes. There are multiple challenges with this type of data, namely the high dimension of the data features. However, we see from Fig. 1.14 that there is significant variance captured in the first few PCA modes. Said another way, the gene data is highly correlated, so that many patients have significant overlap in their gene expression. The ability to visualize patterns and correlations in high-dimensional data is an important reason to use PCA, and PCA has been widely used to find patterns in high-dimensional biological and genetic data [586].

More importantly, patients with ovarian cancer appear to cluster separately from patients without cancer when plotted in the space spanned by the first three PCA modes. This is shown in Fig. 1.15, which is generated by Code 1.5. This inherent clustering in PCA space of data by category is a foundational element of machine learning and pattern recognition. For example, we will see in Section 1.6 that images of different human faces will form clusters in PCA space. The use of these clusters will be explored in greater detail in Chapter 5.

Code 1.5 [MATLAB] Compute PCA for ovarian cancer data.

```
load ovariancancer; % Load ovarian cancer data
[U,S,V] = svd(obs,'econ');
for i=1:size(obs,1)
    x = V(:,1)'*obs(i,:);
    y = V(:,2)'*obs(i,:);
    z = V(:,3)'*obs(i,:);
```

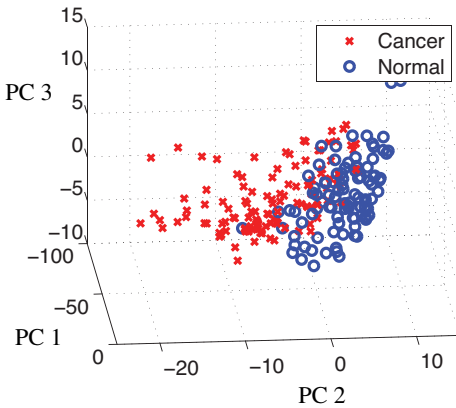


Figure 1.15 Clustering of samples that are normal and those that have cancer in the first three principal component coordinates.

```

    if (grp{i} == 'Cancer')
        plot3(x,y,z, 'rx', 'LineWidth', 2);
    else
        plot3(x,y,z, 'bo', 'LineWidth', 2);
    end
end

```

Code 1.5 [Python] Compute PCA for ovarian cancer data.

```

obs = np.loadtxt(os.path.join('.', 'DATA', 'ovariancancer_obs.csv'), delimiter=',')
f = open(os.path.join('.', 'DATA', 'ovariancancer_grp.csv'), 'r')
grp = f.read().split("\n")

U, S, VT = np.linalg.svd(obs, full_matrices=0)
for j in range(obs.shape[0]):
    x = VT[0,:] @ obs[j,:].T
    y = VT[1,:] @ obs[j,:].T
    z = VT[2,:] @ obs[j,:].T

    if grp[j] == 'Cancer':
        ax.scatter(x,y,z, marker='x', color='r', s=50)
    else:
        ax.scatter(x,y,z, marker='o', color='b', s=50)

```

1.6 Eigenfaces Example

One of the most striking demonstrations of SVD/PCA is the so-called eigenfaces example. In this problem, PCA (i.e., SVD on mean-subtracted data) is applied to a large library of facial images to extract the most dominant correlations between images. The result of this decomposition is a set of *eigenfaces* that define a new coordinate system. Images may be represented in these coordinates by taking the dot product with each of the principal components. It will be shown in [Chapter 5](#) that images of the same person tend to cluster in the eigenface space, making this a useful transformation for facial recognition and



Figure 1.16 (left) A single image for each person in the Yale database, and (right) all images for a specific person. Left panel generated by [Code 1.6](#).

classification [67, 685]. The eigenface problem was first studied by Sirovich and Kirby in 1987 [662] and expanded on in [387]. Its application to automated facial recognition was presented by Turk and Pentland in 1991 [726].

Here, we demonstrate this algorithm using the Extended Yale Face Database B [273], consisting of cropped and aligned images [434] of 38 individuals (28 from the extended database, and 10 from the original database) under nine poses and 64 lighting conditions.⁶ Each image is 192 pixels tall and 168 pixels wide. Unlike the previous image example in [Section 1.2](#), each of the facial images in our library has been reshaped into a large column vector with $192 \times 168 = 32\,256$ elements. We use the first 36 people in the database (left panel of [Fig. 1.16](#)) as our training data for the eigenfaces example, and we hold back two people as a test set. An example of all 64 images of one specific person is shown in the right panel. These images are loaded and plotted using [Code 1.6](#).

Code 1.6 [MATLAB] Plot image for each person in the Yale database ([Fig. 1.16](#)).

```
load ../DATA/allFaces.mat

allPersons = zeros(n*6,m*6); % Make array to fit all faces
count = 1;
for i=1:6 % 6 x 6 grid of faces
    for j=1:6
        allPersons(1+(i-1)*n:i*n,1+(j-1)*m:j*m) ...
            =reshape(faces(:,1+sum(nfaces(1:count-1))),n,m);
        count = count + 1;
    end
end
imagesc(allPersons), colormap gray
```

⁶ The Yale database can be downloaded at <http://vision.ucsd.edu/~iskwak/ExtYaleDatabase/ExtYaleB.html>.

Code 1.6 [Python] Plot image for each person in the Yale database (Fig. 1.16).

```
mat_contents = scipy.io.loadmat(os.path.join('.', 'DATA', '
    allFaces.mat'))
faces = mat_contents['faces']
m = int(mat_contents['m'])
n = int(mat_contents['n'])
nfaces = np.ndarray.flatten(mat_contents['nfaces'])

allPersons = np.zeros((n*6,m*6))
count = 0

for j in range(6):
    for k in range(6):
        allPersons[j*n : (j+1)*n, k*m : (k+1)*m] = np.
            reshape(faces[:, np.sum(nfaces[:count])], (m,n)).T
        count += 1

img = plt.imshow(allPersons)
```

As mentioned before, each image is reshaped into a large column vector, and the average face is computed and subtracted from each column vector. The mean-subtracted image vectors are then stacked horizontally as columns in the data matrix $\mathbf{X}$, as shown in Fig. 1.17. Thus, taking the SVD of the mean-subtracted matrix $\mathbf{X}$ results in the PCA. The columns of $\mathbf{U}$ are the eigenfaces, and they may be reshaped back into 192×168 images. This is illustrated in Code 1.7.

Code 1.7 [MATLAB] Compute eigenfaces on mean-subtracted data.

```
% We use the first 36 people for training data
trainingFaces = faces(:,1:sum(nfaces(1:36)));
avgFace = mean(trainingFaces,2); % size n*m by 1;

% Compute eigenfaces on mean-subtracted training data
X = trainingFaces - avgFace*ones(1,size(trainingFaces,2));
[U,S,V] = svd(X,'econ');

imagesc(reshape(avgFace,n,m)) % Plot avg face
imagesc(reshape(U(:,1),n,m)) % Plot first eigenface
```

Code 1.7 [Python] Compute eigenfaces on mean-subtracted data.

```
# We use the first 36 people for training data
trainingFaces = faces[:, np.sum(nfaces[:36])]
avgFace = np.mean(trainingFaces, axis=1) # size n*m by 1

# Compute eigenfaces on mean-subtracted training data
X = trainingFaces - np.tile(avgFace, (trainingFaces.shape
    [1], 1)).T
U, S, VT = np.linalg.svd(X, full_matrices=0)
img_avg = ax1.imshow(np.reshape(avgFace, (m,n)).T)
img_u1 = ax2.imshow(np.reshape(U[:, 0], (m,n)).T)
```

Using the eigenfaces library, $\tilde{\mathbf{U}}$, obtained above, we now attempt to approximately represent an image that was not in the training data. At the beginning, we held back two

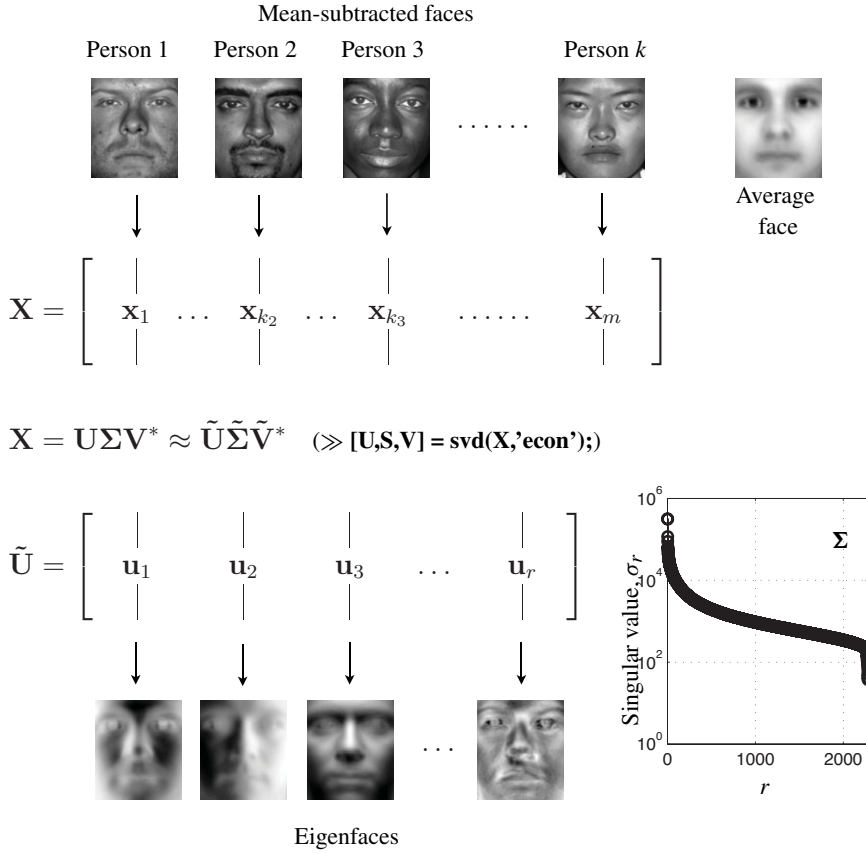


Figure 1.17 Schematic procedure to obtain eigenfaces from library of faces X after subtracting off average face $\bar{X}$.

individuals (the 37th and 38th people), and we now use one of their images as a test image, x_{test} . We will see how well a rank- r SVD basis will approximate this image using the following projection:

$$\tilde{x}_{\text{test}} = \tilde{U}\tilde{U}^* x_{\text{test}}.$$

The eigenface approximation for various values of r is shown in [Fig. 1.18](#), as computed using [Code 1.8](#). The approximation is relatively poor for $r \leq 200$, although for $r > 400$ it converges to a passable representation of the test image.

It is interesting to note that the eigenface space is not only useful for representing human faces, but may also be used to approximate a dog ([Fig. 1.19](#)) or a cappuccino ([Fig. 1.20](#)). This is possible because the 1600 eigenfaces span a large subspace of the 32 256-dimensional image space corresponding to broad, smooth, non-localized spatial features, such as cheeks, forehead, mouths, etc.

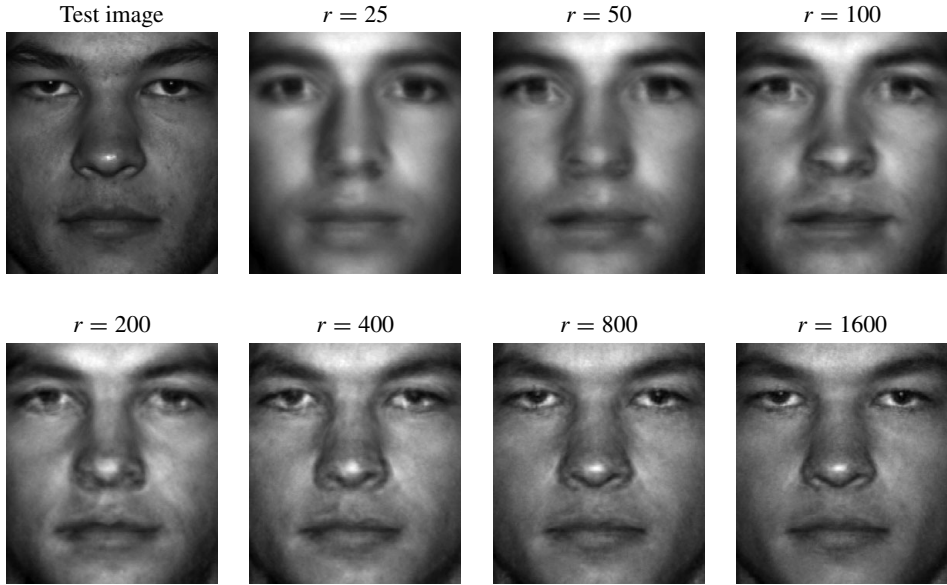


Figure 1.18 Approximate representation of test image using eigenfaces basis of various order r . Test image is not in training set.

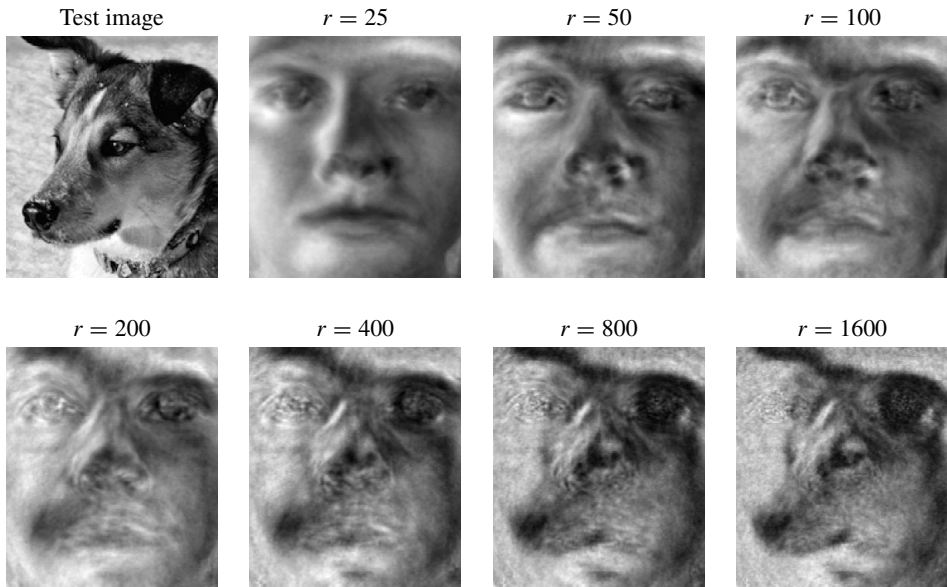


Figure 1.19 Approximate representation of an image of a dog using eigenfaces.

Code 1.8 [MATLAB] Approximate test image omitted from training data.

```
testFace = faces(:, 1+sum(nfaces(1:36))); % Person 37
testFaceMS = testFace - avgFace;
for r=[25 50 100 200 400 800 1600]
```

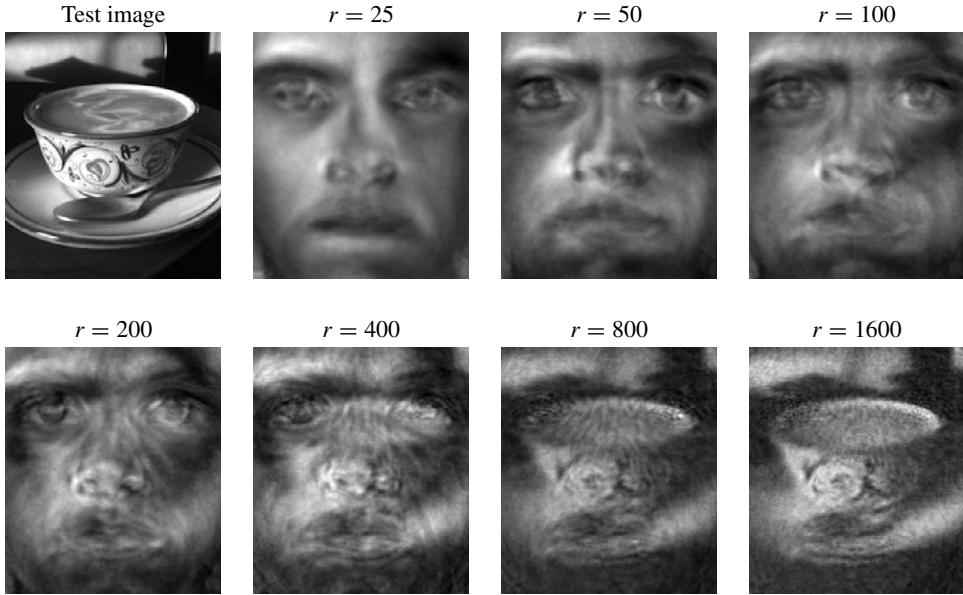


Figure 1.20 Approximate representation of a cappuccino using eigenfaces.

```
reconFace = avgFace + (U(:,1:r)*(U(:,1:r)'*testFaceMS));
imagesc(reshape(reconFace,n,m))
end
```

Code 1.8 [Python] Approximate test image omitted from training data.

```
testFace = faces[:,np.sum(nfaces[:36])] # Person 37
testFaceMS = testFace - avgFace
r_list = [25, 50, 100, 200, 400, 800, 1600]
for r in r_list:
    reconFace = avgFace + U[:, :r] @ U[:, :r].T @ testFaceMS
    img = plt.imshow(np.reshape(reconFace, (m,n)).T)
```

We further investigate the use of the eigenfaces as a coordinate system, defining an eigenface space. By projecting an image $\mathbf{x}$ onto the first r PCA modes, we obtain a set of coordinates in this space: $\tilde{\mathbf{x}} = \tilde{\mathbf{U}}^* \mathbf{x}$. Some principal components may capture the most common features shared among all human faces, while other principal components will be more useful for distinguishing between individuals. Additional principal components may capture differences in lighting angles. [Figure 1.21](#) shows the coordinates of all 64 images of two individuals projected onto the 5th and 6th principal components, generated by [Code 1.9](#). Images of the two individuals appear to be well separated in these coordinates. This is the basis for image recognition and classification in [Chapter 5](#).

Code 1.9 [MATLAB] Project images for two specific people onto the 5th and 6th eigenfaces to illustrate the potential for automated classification.

```
P1num = 2; % Person number 2
P2num = 7; % Person number 7

P1 = faces(:,1+sum(nfaces(1:P1num-1)):sum(nfaces(1:P1num)) );
P2 = faces(:,1+sum(nfaces(1:P2num-1)):sum(nfaces(1:P2num)) );
```

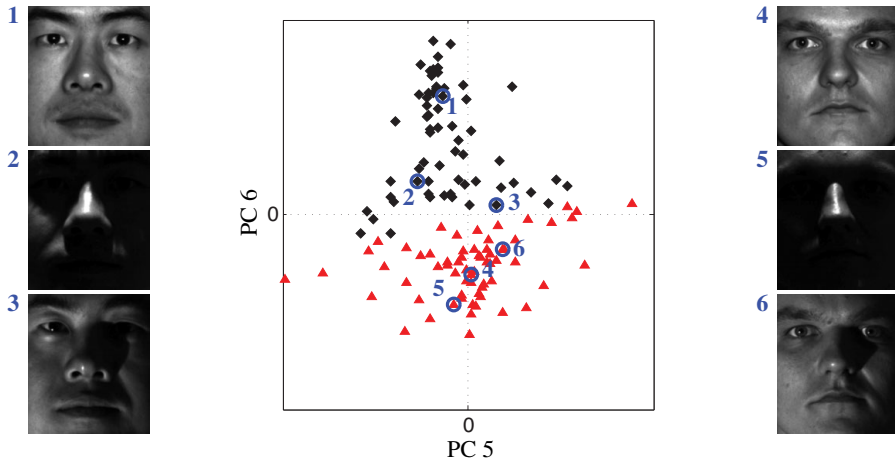


Figure 1.21 Projection of all images from two individuals onto the 5th and 6th PCA modes. Projected images of the first individual are indicated with black diamonds, and projected images of the second individual are indicated with red triangles. Three examples from each individual are circled in blue, and the corresponding image is shown.

```
P1 = P1 - avgFace*ones(1,size(P1,2));
P2 = P2 - avgFace*ones(1,size(P2,2));

PCAmodes = [5 6]; % Project onto PCA modes 5 and 6
PCACoordsP1 = U(:,PCAmodes)'*P1;
PCACoordsP2 = U(:,PCAmodes)'*P2;

plot(PCACoordsP1(1,:),PCACoordsP1(2:,:), 'kd'), hold on
plot(PCACoordsP2(1,:),PCACoordsP2(2:,:), 'r^')
```

Code 1.9 [Python] Project images for two specific people onto the 5th and 6th eigenfaces to illustrate the potential for automated classification.

```
P1num = 2 # Person number 2
P2num = 7 # Person number 7

P1 = faces[:,np.sum(nfaces[: (P1num-1)]) : np.sum(nfaces[: P1num
])]]
P2 = faces[:,np.sum(nfaces[: (P2num-1)]) : np.sum(nfaces[: P2num
])]]

P1 = P1 - np.tile(avgFace, (P1.shape[1],1)).T
P2 = P2 - np.tile(avgFace, (P2.shape[1],1)).T

PCAmodes = [5, 6] # Project onto PCA modes 5 and 6
PCACoordsP1 = U[:,PCAmodes-np.ones_like(PCAmodes)].T @ P1
PCACoordsP2 = U[:,PCAmodes-np.ones_like(PCAmodes)].T @ P2

plt.plot(PCACoordsP1[0,:],PCACoordsP1[1,:], 'd',Color='k')
plt.plot(PCACoordsP2[0,:],PCACoordsP2[1,:], '^',Color='r')
```

1.7 Truncation and Alignment

Deciding how many singular values to keep, i.e., where to truncate, is one of the most important and contentious decisions when using the SVD. There are many factors, including specifications on the desired rank of the system, the magnitude of noise, and the distribution of the singular values. Often, one truncates the SVD at a rank r that captures a predetermined amount of the variance or energy in the original data, such as 90% or 99% truncation. Although crude, this technique is commonly used. Other techniques involve identifying “elbows” or “knees” in the singular value distribution, which may denote the transition from singular values that represent important patterns from those that represent noise. Truncation may be viewed as a hard threshold on singular values, where values larger than a threshold τ are kept, while remaining singular values are truncated. Recent work by Gavish and Donoho [266] provides an optimal truncation value, or hard threshold, under certain conditions, providing a principled approach to obtaining low-rank matrix approximations using the SVD.

In addition, the alignment of data significantly impacts the rank of the SVD approximation. The SVD essentially relies on a separation of variables between the columns and rows of a data matrix. In many situations, such as when analyzing traveling waves or misaligned data, this assumption breaks down, resulting in an artificial rank inflation.

Optimal Hard Threshold

A recent theoretical breakthrough determines the *optimal* hard threshold τ for singular value truncation under the assumption that a matrix has a low-rank structure contaminated with Gaussian white noise [266]. This work builds on a significant literature surrounding various techniques for hard and soft thresholding of singular values. In this section, we summarize the main results and demonstrate the thresholding on various examples. For more details, see [266].

First, we assume that the data matrix $\mathbf{X}$ is the sum of an underlying low-rank, or approximately low-rank, matrix $\mathbf{X}_{\text{true}}$ and a noise matrix $\mathbf{X}_{\text{noise}}$:

$$\mathbf{X} = \mathbf{X}_{\text{true}} + \gamma \mathbf{X}_{\text{noise}}. \quad (1.43)$$

The entries of $\mathbf{X}_{\text{noise}}$ are assumed to be independent, identically distributed (i.i.d.) Gaussian random variables with zero mean and unit variance. The magnitude of the noise is characterized by γ , which deviates from the notation in [266].⁷

When the noise magnitude γ is known, there are closed-form solutions for the optimal hard threshold τ :

1. If $\mathbf{X} \in \mathbb{R}^{n \times n}$ is square, then

$$\tau = (4/\sqrt{3})\sqrt{n}\gamma. \quad (1.44)$$

2. If $\mathbf{X} \in \mathbb{R}^{n \times m}$ is rectangular and $m \ll n$, then the constant $4/\sqrt{3}$ is replaced by a function of the aspect ratio $\beta = m/n$:

$$\tau = \lambda(\beta)\sqrt{n}\gamma, \quad (1.45)$$

⁷ In [266], σ is used to denote standard deviation and y_k denotes the k th singular value.

$$\lambda(\beta) = \left(2(\beta + 1) + \frac{8\beta}{(\beta + 1) + (\beta^2 + 14\beta + 1)^{1/2}} \right)^{1/2}. \quad (1.46)$$

Note that this expression reduces to (1.44) when $\beta = 1$. If $n \ll m$, then $\beta = n/m$.

When the noise magnitude γ is unknown, which is more typical in real-world applications, then it is possible to estimate the noise magnitude and scale the distribution of singular values by using σ_{med} , the *median* singular value. In this case, there is no closed-form solution for τ , and it must be approximated numerically:

3. For unknown noise γ , and a rectangular matrix $\mathbf{X} \in \mathbb{R}^{n \times m}$, the optimal hard threshold is given by

$$\tau = \omega(\beta)\sigma_{\text{med}}. \quad (1.47)$$

Here, $\omega(\beta) = \lambda(\beta)/\mu_\beta$, where μ_β is the solution to the following problem:

$$\int_{(1-\beta)^2}^{\mu_\beta} \frac{\{[(1 + \sqrt{\beta})^2 - t][t - (1 - \sqrt{\beta})^2]\}^{1/2}}{2\pi t} dt = \frac{1}{2}.$$

Solutions to the expression above must be approximated numerically. Fortunately [266] has a MATLAB code supplement⁸ [205] to approximate μ_β .

The new method of optimal hard thresholding works remarkably well, as demonstrated on the examples below.

Example 1: Toy Problem

In the first example, shown in Fig. 1.22 and Code 1.10, we artificially construct a rank-two matrix and contaminate the data with Gaussian white noise. A de-noised and dimensionally reduced matrix is then obtained using the threshold from (1.44), as well as a truncation keeping 90% of the cumulative sum of singular values. It is clear that the hard threshold is able to filter the noise more effectively. Plotting the singular values in Fig. 1.23, it is clear that there are two values that are above threshold.

Code 1.10 [MATLAB] Compare various thresholding approaches on noisy low-rank data (Fig. 1.22).

```
% Generate underlying low-rank data
t = (-3:.01:3)';

Utrue = [cos(17*t).*exp(-t.^2) sin(11*t)];
Strue = [2 0; 0 .5];
Vtrue = [sin(5*t).*exp(-t.^2) cos(13*t)];

X = Utrue*Strue*Vtrue';
figure, imshow(X);

% Contaminate signal with noise
sigma = 1;
Xnoisy = X+sigma*randn(size(X));
figure, imshow(Xnoisy);
```

⁸ <http://purl.stanford.edu/vg705qn9070>